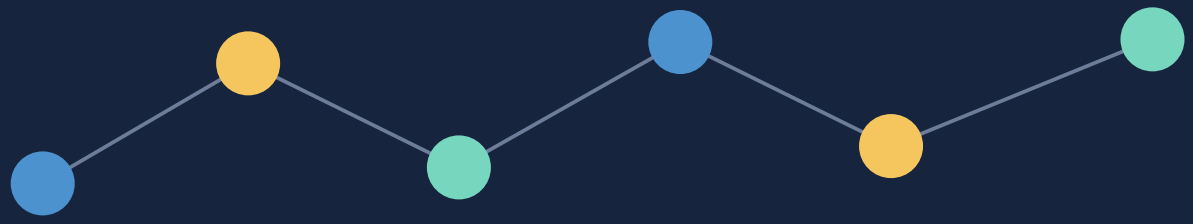
GATE 2027 / COMPLETE SYLLABUS EDITION

# Theory of Computation

Start from the basics. Build every syllabus idea. Then practise.

Section 6: Theory of Computation



LEARN → SEE → WORK THROUGH → RETRIEVE → PRACTISE

**1 Understand**

Each chapter starts in plain language.

**2 Visualize**

Follow the flow and redraw it.

**3 Work**

Trace the worked example with pencil.

**4 Recall**

Answer before checking the solution.

A beginner learning guide + illustrated recall/practice + your supplied reference material

# 2027 syllabus coverage map

Use this as a route map: every item below has a beginner lesson in this PDF. The supplied handbook/chapter PDFs are gathered in the reference section after the learning and practice pages.

- [ ] **Regular expressions and finite automata**

  Regex operations • DFA • NFA • Conversion

- [ ] **Context-free grammars and push-down automata**

  Grammar pieces • Derivations and ambiguity • PDA stack • CFG–PDA link

- [ ] **Regular and context-free languages**

  Regular language • Context-free language • Memory comparison • Closure warning

- [ ] **Pumping lemmas: regular and context-free**

  Regular-language lemma • Proof order • Context-free lemma • What the lemma cannot prove

- [ ] **Turing machines and undecidability**

  Machine model • Decidable language • Recognizable language • Undecidability

- [ ] **Regular languages: closures, products and minimal DFAs**

  Closure tests • Product construction • Modulo counters • Distinguishable states

- [ ] **CFG questions: parse trees, PDA power and decidability**

  Ambiguity • PDA and CFL • Decidable CFG properties • Pumping lemma role

# Your route through this subject

Follow the lessons in order. Tick a topic only after you can explain its picture without looking; a tick means “I can teach it,” not “I have seen it.”


1. Regular expressions and finite automata
2. Context-free grammars and push-down automata
3. Regular and context-free languages
4. Pumping lemmas: regular and context-free
5. Turing machines and undecidability
6. Regular languages: closures, products and minimal DFAs
7. CFG questions: parse trees, PDA power and decidability


## THE 20-MINUTE LEARNING LOOP

| 5 min | 5 min | 7 min | 3 min |
|---|---|---|---|
| Read | Redraw | Solve | Recall |

**IF THIS SUBJECT FEELS COMPLETELY NEW**

That is okay. Start with the first lesson, keep a pencil beside you, and redraw its picture once. Attempt the recall prompt before opening the answer guide.

# Regular expressions and finite automata

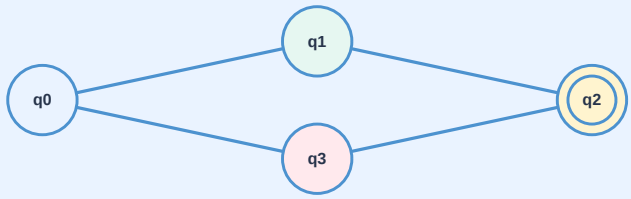
### START WITH THE IDEA

A language is a set of strings over an alphabet. Regular expressions describe simple pattern families; finite automata recognize exactly the regular languages.

### A CONCRETE FIRST EXAMPLE

(0|1)* describes every binary string. 01(0|1)* describes strings beginning with 01.

### PICTURE IT




### IMAGINE IT THIS WAY

A finite automaton is a tiny doorman with a finite number of moods and no notebook.

## THE IDEAS, ONE BY ONE

**Regex operations**

Union chooses alternatives, concatenation joins strings, and star repeats zero or more times. epsilon denotes the empty string.

**DFA**

One state at a time; every state-symbol pair has exactly one transition.

**NFA**

May branch or use epsilon-transitions. It recognizes no more languages than a DFA, though its state diagram may be smaller.

**Conversion**

Subset construction represents a DFA state as a set of NFA states; accepting if the set contains an accepting NFA state.

### HOW TO USE IT ON A PROBLEM

State the language property being tested. Choose the matching machine or proof tool, trace its states/stack/split carefully, and respect every quantifier.

**WATCH OUT** The empty string epsilon is not the empty language empty set.

### TRY FROM MEMORY

Cover this page. Explain Regular expressions and finite automata using the picture, then give one example before looking again.

# Context-free grammars and push-down automata

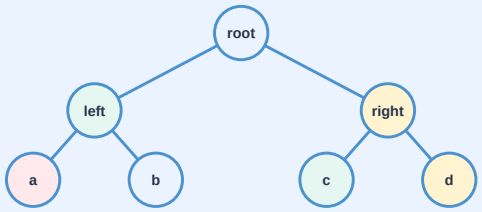
### START WITH THE IDEA

A CFG generates strings by replacing nonterminals; a PDA recognizes patterns using states plus a stack, which can remember nested structure.

### A CONCRETE FIRST EXAMPLE

S -> aSb | epsilon generates epsilon, ab, aabb, aaabbb: each recursive step adds one matching a and b.

### PICTURE IT




### IMAGINE IT THIS WAY

A PDA's stack is a pile of unmatched opening brackets.

## THE IDEAS, ONE BY ONE

**Grammar pieces**

Terminals appear in output; nonterminals are variables; productions describe replacements; the start symbol begins derivation.

**Derivations and ambiguity**

A derivation applies productions. A string is ambiguous if it has multiple parse trees under the grammar.

**PDA stack**

Push/pop symbols provide unbounded last-in-first-out memory, useful for matching nested delimiters.

**CFG–PDA link**

CFGs and nondeterministic push-down automata characterize context-free languages.

### HOW TO USE IT ON A PROBLEM

State the language property being tested. Choose the matching machine or proof tool, trace its states/stack/split carefully, and respect every quantifier.

**WATCH OUT** A finite automaton cannot count arbitrary nesting depth with only finite states.

### TRY FROM MEMORY

Cover this page. Explain Context-free grammars and push-down automata using the picture, then give one example before looking again.

# Regular and context-free languages

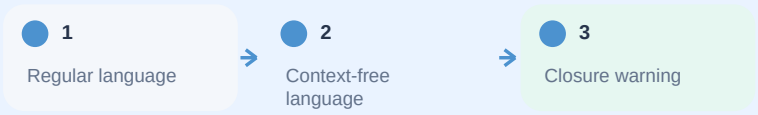
### START WITH THE IDEA

Language families describe which string patterns a machine can recognize. Finite automata have only a fixed number of states; push-down automata add a stack, which can grow as the input grows.

### A CONCRETE FIRST EXAMPLE

Balanced parentheses are context-free: push for each opening bracket and pop for each closing bracket. The language {a^n b^n : n >= 0} is also context-free, but a finite automaton cannot match arbitrarily many a symbols to b symbols.

### PICTURE IT




### IMAGINE IT THIS WAY

A DFA has a handful of sticky notes; a PDA has those notes plus an expandable stack of plates.

## THE IDEAS, ONE BY ONE

**Regular language**

A language recognized by a DFA or NFA. Regular languages are closed under union, intersection, complement, concatenation and star.

**Context-free language**

A language generated by a CFG or recognized by a nondeterministic PDA; a stack supports nested patterns such as balanced parentheses.

**Memory comparison**

A finite automaton cannot remember an unbounded count. A PDA can push one marker for each opening symbol and pop it for each matching close.

**Closure warning**

CFLs are closed under union and concatenation, but not in general under intersection or complement.

### HOW TO USE IT ON A PROBLEM

State the language property being tested. Choose the matching machine or proof tool, trace its states/stack/split carefully, and respect every quantifier.

**WATCH OUT** “Context-free” does not mean a string has no context; it names a grammar family.

### TRY FROM MEMORY

Cover this page. Explain Regular and context-free languages using the picture, then give one example before looking again.

# Pumping lemmas: regular and context-free

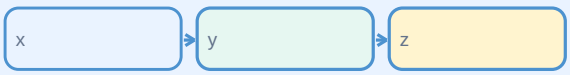
### START WITH THE IDEA

A pumping lemma states a pattern that every sufficiently long string in a language from that family must allow. To prove a language is outside the family, choose a string and show every legal split fails after pumping.

### A CONCRETE FIRST EXAMPLE

Assume {a^n b^n : n >= 0} is regular with pumping length p. Choose w=a^p b^p. Since |xy| <= p, the nonempty y contains only a symbols: y=a^k for some k>0. Pump with i=0: xz=a^(p-k)b^p, which has unequal counts, contradicting membership.

### PICTURE IT




remove / repeat the middle

### IMAGINE IT THIS WAY

The regular-language split exposes a loop among the first p steps of a finite-state machine. Pumping repeats or removes that loop.

## THE IDEAS, ONE BY ONE

**Regular-language lemma**

For a regular language with pumping length p, every string w in the language with |w| >= p has a split w=xyz with |xy| <= p and |y| > 0, such that x y^i z remains in the language for every integer i >= 0.

**Proof order**

Choose w after p is given; keep the split arbitrary; use the length rule to constrain y; choose an allowed i that breaks membership. This proves no legal split works.

**Context-free lemma**

For a sufficiently long CFL string w=u v w0 x y, |v w0 x| <= p and |v x| > 0; u v^i w0 x^i y must remain in the language for every i >= 0.

**What the lemma cannot prove**

Failure of a pumping property proves non-membership. Satisfying a pumping property does not prove that a language is regular or context-free.

### HOW TO USE IT ON A PROBLEM

State the language property being tested. Choose the matching machine or proof tool, trace its states/stack/split carefully, and respect every quantifier.

**WATCH OUT** The quantifier order matters: choose w, then handle every legal split, then select i. Do not pick a convenient split yourself.

### TRY FROM MEMORY

Cover this page. Explain Pumping lemmas: regular and context-free using the picture, then give one example before looking again.

# Turing machines and undecidability

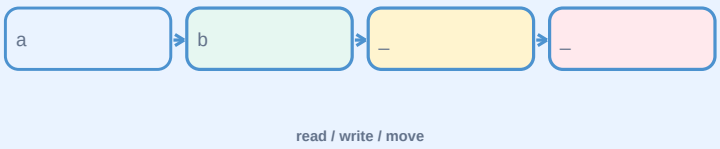
### START WITH THE IDEA

A Turing machine has finite control and an unbounded tape. It models general computation; some well-defined questions have no algorithm that always returns the correct yes/no answer.

### A CONCRETE FIRST EXAMPLE

A program that decides halting for every program can be used to build a program that contradicts its own prediction; hence no such universal decider exists.

### PICTURE IT




read / write / move

### IMAGINE IT THIS WAY

The tape is an unlimited notebook; finite control describes the instruction rule.

## THE IDEAS, ONE BY ONE

**Machine model**

A transition reads the current symbol, writes one, changes state and moves the head.

**Decidable language**

A decider halts on every input and accepts exactly members.

**Recognizable language**

A recognizer accepts members; on nonmembers it may reject or loop forever.

**Undecidability**

Diagonal/self-reference arguments show limits. The halting problem asks whether an arbitrary program stops on an input and is undecidable.

### HOW TO USE IT ON A PROBLEM

State the language property being tested. Choose the matching machine or proof tool, trace its states/stack/split carefully, and respect every quantifier.

**WATCH OUT** Distinguish “not decidable” from “not recognizable.”

### TRY FROM MEMORY

Cover this page. Explain Turing machines and undecidability using the picture, then give one example before looking again.

# Regular languages: closures, products and minimal DFAs

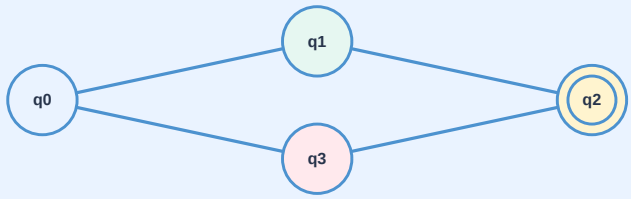
### START WITH THE IDEA

A DFA remembers only finite information about the input. Product states let it track two properties at once; minimizing merges states that no future suffix can distinguish.

### A CONCRETE FIRST EXAMPLE

To recognize strings where the number of a symbols is 2 mod 7 and b symbols is 3 mod 9, track the pair of remainders; at most 7x9 states are needed before minimization.

### PICTURE IT




### IMAGINE IT THIS WAY

A DFA is a tiny notebook with a finite number of boxes. A modulo counter only needs to remember its remainder.

## THE IDEAS, ONE BY ONE

**Closure tests**

Regular languages are closed under union, intersection, complement, difference, concatenation and star.

**Product construction**

To track two DFAs, use paired states (p,q). Mark a pair accepting according to the required Boolean combination.

**Modulo counters**

A fixed modulus needs finitely many remainder states. Combine counters by taking their product state.

**Distinguishable states**

Two states differ if some suffix is accepted from one and rejected from the other. Such states cannot be merged in a minimum DFA.

### HOW TO USE IT ON A PROBLEM

State the language property being tested. Choose the matching machine or proof tool, trace its states/stack/split carefully, and respect every quantifier.

**WATCH OUT** Finite memory can count modulo a fixed number, but cannot store an unbounded exact count.

### TRY FROM MEMORY

Cover this page. Explain Regular languages: closures, products and minimal DFAs using the picture, then give one example before looking again.

# CFG questions: parse trees, PDA power and decidability

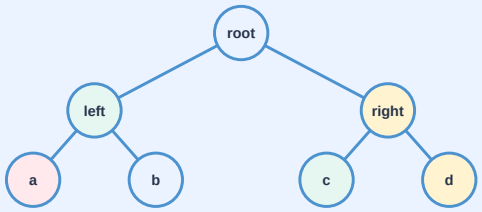
### START WITH THE IDEA

A context-free grammar can nest matching structure with a stack. Some language questions have algorithms; others cannot be decided for every possible grammar or machine.

### A CONCRETE FIRST EXAMPLE

For S -> aSb | epsilon, strings have equal numbers of a and b in matching order; a stack can push each a and pop each b.

### PICTURE IT




### IMAGINE IT THIS WAY

A stack is a pile of plates: it remembers nested obligations in last-in, first-out order.

## THE IDEAS, ONE BY ONE

**Ambiguity**

A grammar is ambiguous if some string has two distinct parse trees or leftmost derivations.

**PDA and CFL**

A pushdown automaton has a stack and recognizes context-free languages. Every regular language is context-free; not every CFL is deterministic context-free.

**Decidable CFG properties**

Emptiness and membership for a fixed string are decidable. Equivalence of arbitrary CFGs is undecidable.

**Pumping lemma role**

Use the lemma to prove a language is not regular (or not context-free in the CFL version); it does not prove regularity.

### HOW TO USE IT ON A PROBLEM

State the language property being tested. Choose the matching machine or proof tool, trace its states/stack/split carefully, and respect every quantifier.

**WATCH OUT** Do not claim every CFL is recognized by a deterministic PDA.

### TRY FROM MEMORY

Cover this page. Explain CFG questions: parse trees, PDA power and decidability using the picture, then give one example before looking again.

# Past-paper trail and coverage check

### WHAT THE PAPERS MATCH

Verified anchors: 2018 Q35-Q37/Q52 (language classes, decidability, lexical regular expressions); 2022 Q12/Q23/Q46-Q48 (regular expressions, closure and decidability/CFLs); 2023 Q13/Q23/Q38-Q39/Q62; 2024 CS1 Q23/Q50/Q61; 2025 CS2 Q24-Q25/Q30/Q52/Q60; 2026 CS1 Q25-Q26/Q51-Q52 and CS2 Q13/Q29/Q47-Q48.

### WHAT THIS EDITION ADDS

Added product automata, modulo counters, state distinguishability, CFG ambiguity, PDA power, decidable-vs-undecidable checks and pumping-lemma limits.

### SYLLABUS BOUNDARY

Do not use the pasted Gemini mapping without the session code: its Q30 closure claim refers to 2025 CS2; 2025 CS1 Q30 is a counting problem about consecutive equal symbols.

### HOW TO USE THIS CHECK

A question is a match when its tested skill is taught in the lesson pages, even if the story or numbers are new. Anchors are selected verified examples, not an exhaustive list; the separate Coverage and Gap Audit PDF identifies years whose scans could not be checked question by question.

# Close the notes: try before you peek

**For each card, explain the idea from memory and recreate the lesson example on paper.**

1. **Regular expressions and finite automata**

   Without notes: define Conversion and explain why this example works. Then recreate the steps on the lesson page.

2. **Context-free grammars and push-down automata**

   Without notes: define CFG–PDA link and explain why this example works. Then recreate the steps on the lesson page.

3. **Regular and context-free languages**

   Without notes: define Closure warning and explain why this example works. Then recreate the steps on the lesson page.

4. **Pumping lemmas: regular and context-free**

   Without notes: define What the lemma cannot prove and explain why this example works. Then recreate the steps on the lesson page.

5. **Turing machines and undecidability**

   Without notes: define Undecidability and explain why this example works. Then recreate the steps on the lesson page.

6. **Regular languages: closures, products and minimal DFAs**

   Without notes: define Distinguishable states and explain why this example works. Then recreate the steps on the lesson page.

# Close the notes: try before you peek

**For each card, explain the idea from memory and recreate the lesson example on paper.**

7

### CFG questions: parse trees, PDA power and decidability

Without notes: define Pumping lemma role and explain why this example works. Then recreate the steps on the lesson page.

# Compare with the worked examples

**Use this only after attempting the recall prompts. The full reasoning is on the numbered lesson page.**

### 1. LESSON 1: Regular expressions and finite automata

(0|1)* describes every binary string. 01(0|1)* describes strings beginning with 01.

### 2. LESSON 2: Context-free grammars and push-down automata

S -> aSb | epsilon generates epsilon, ab, aabb, aaabbb: each recursive step adds one matching a and b.

### 3. LESSON 3: Regular and context-free languages

Balanced parentheses are context-free: push for each opening bracket and pop for each closing bracket. The language {a^n b^n : n >= 0} is also context-free, but a finite automaton cannot match arbitrarily many a symbols to b symbols.

### 4. LESSON 4: Pumping lemmas: regular and context-free

Assume {a^n b^n : n >= 0} is regular with pumping length p. Choose w=a^p b^p. Since |xy| <= p, the nonempty y contains only a symbols: y=a^k for some k>0. Pump with i=0: xz=a^(p-k)b^p, which has unequal counts, contradicting membership.

### 5. LESSON 5: Turing machines and undecidability

A program that decides halting for every program can be used to build a program that contradicts its own prediction; hence no such universal decider exists.

### 6. LESSON 6: Regular languages: closures, products and minimal DFAs

To recognize strings where the number of a symbols is 2 mod 7 and b symbols is 3 mod 9, track the pair of remainders; at most 7x9 states are needed before minimization.

# Compare with the worked examples

**Use this only after attempting the recall prompts. The full reasoning is on the numbered lesson page.**

**7. LESSON 7: CFG questions: parse trees, PDA power and decidability**

For S -> aSb | epsilon, strings have equal numbers of a and b in matching order; a stack can push each a and pop each b.

# Your practice and reference library

01 **Beginner lessons**

Plain-language explanation, visual sequence, example and watch-outs for every listed 2027 topic cluster.

02 **Recall and GATE-style practice**

The illustrated recap section includes questions and a separate answer explanation.

03 **Detailed reference**

Your supplied subject handbook and chapter PDFs follow for extra depth, reading and worked material.

## Included source documents

- **Illustrated recall and GATE-style practice** — TOC - Theory of Computation - GATE 2027 Illustrated Notes.pdf — 11 pages
- **Detailed supplied reference** — ToC.pdf — 42 pages

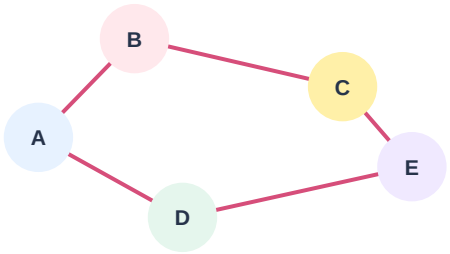
The official 2027 syllabus map precedes the lessons. Original reference pages are retained so you can use them for slower reading and extra practice.

# Theory of Computation

## GATE 2027 | VISUAL LEARNING NOTES | STARTING FROM THE BASICS

Classify the language. Choose the machine. Prove the limit.

### PICTURE IT




pattern -> machine

### HOW TO USE THIS NOTEBOOK

1. **Learn the idea**
   Read the plain-language explanation before memorizing a formula.
2. **See it**
   Study the labeled picture and redraw it from memory.
3. **Follow a solve**
   Work through the examples step by step.
4. **Test yourself**
   Attempt the GATE-style questions before opening the key.

### THE PATH YOU WILL FOLLOW

- Regular languages
- Context-free languages
- Computability

### IF YOU FEEL COMPLETELY NEW

That is okay. Start with the first topic page, keep a pencil beside you, and copy each diagram once. Do not jump straight to the answer key. The questions are meant to come after the concepts.

# Your route through this subject

Follow the topics in order. Each page teaches one idea, shows a picture, gives a recall cue and points out a common trap.

## REGULAR LANGUAGES

- [ ] Regular expressions.
- [ ] Finite automata.

## CONTEXT-FREE LANGUAGES

- [ ] Context-free grammars.
- [ ] Push-down automata.
- [ ] Regular and context-free languages; pumping lemma.

## COMPUTABILITY

- [ ] Turing machines.
- [ ] Undecidability.

### THE 20-MINUTE LEARNING LOOP

| 5 min | 5 min | 7 min | 3 min |
|---|---|---|---|
| Read | Redraw | Solve | Recall |

Tick a topic only after you can explain its picture without looking. A tick means "I can teach it," not "I have seen it."

### BEFORE EACH STUDY SESSION

Pick one topic. Read its idea, redraw its diagram, follow the worked examples, then answer questions with the notes closed. Circle only the step where you got stuck - that is your next revision target.

CONCEPT 01 / TOC

# Regular expressions and languages

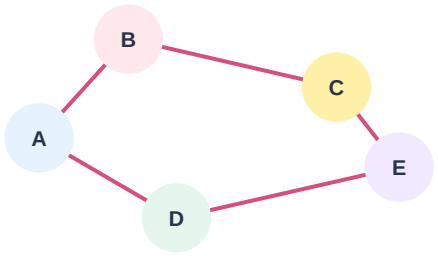
### START WITH THE IDEA

Regular expressions describe sets using union, concatenation and Kleene star. Regular languages are closed under union, intersection, complement, concatenation and star. A DFA/NFA recognizes exactly the regular languages.

### A CONCRETE FIRST EXAMPLE

A regular expression describes a set of strings using operations such as union, concatenation and repetition. Finite automata recognize exactly the regular languages. (0|1)* describes every binary string, including the empty string; 01(0|1)* describes strings beginning with 01.

## PICTURE IT




pattern -> machine

### IMAGINE IT THIS WAY

A regex is a route recipe; star means repeat the route zero or more times.

### HOW TO USE IT ON A PROBLEM

Translate small strings into examples and counterexamples.

### WATCH OUT

A regex pattern is not the same thing as a programming-language regex engine with extensions.

### TRY FROM MEMORY

Cover the page and explain Regular expressions and languages using the diagram. Give one example before looking again.

CONCEPT 02 / TOC

# DFA, NFA and subset construction

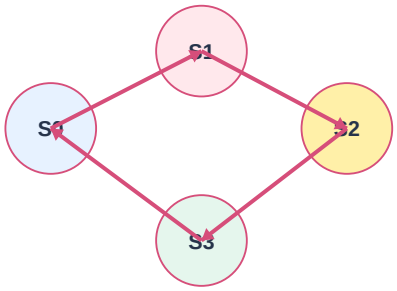
### START WITH THE IDEA

A DFA has one next state per state/symbol. An NFA may have several choices and epsilon moves. Convert NFA to DFA by making each DFA state a set of NFA states and applying epsilon-closure.

### A CONCRETE FIRST EXAMPLE

A DFA has exactly one next state for every state-symbol pair. An NFA may branch or use epsilon-moves, but subset construction converts its possible-state sets into DFA states. If an NFA can be in {q0,q2} after reading a prefix, the DFA's corresponding state is the set {q0,q2}.

## PICTURE IT




state + event -> next state

### IMAGINE IT THIS WAY

An NFA is a team of scouts; a DFA state records the set of scouts still possible.

### HOW TO USE IT ON A PROBLEM

For each subset and symbol: move, then epsilon-close.

### WATCH OUT

An NFA can be smaller; it does not recognize more languages than a DFA.

### TRY FROM MEMORY

Cover the page and explain DFA, NFA and subset construction using the diagram. Give one example before looking again.

CONCEPT 03 / TOC

# CFGs and push-down automata

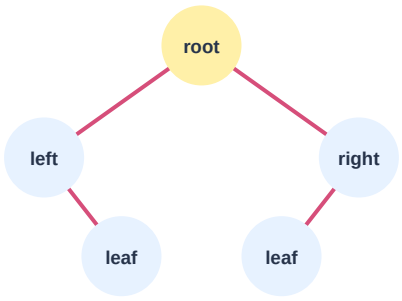
### START WITH THE IDEA

A CFG replaces a nonterminal by a production. A PDA adds a stack, enabling nested structure. Every regular language is context-free, but not every CFL is regular.

### A CONCRETE FIRST EXAMPLE

A context-free grammar replaces a nonterminal with a production; a PDA adds a stack to remember nested structure. This extra memory handles balanced parentheses, which a finite automaton cannot count without bound. S -> (S)S | epsilon generates balanced parentheses: S⇒(S)S⇒()S⇒().

## PICTURE IT




### IMAGINE IT THIS WAY

A stack lets a machine remember matching openings, like parentheses.

### HOW TO USE IT ON A PROBLEM

Trace derivations and stack contents step by step.

### WATCH OUT

A PDA stack is unbounded in theory; a finite automaton has only finite state.

### TRY FROM MEMORY

Cover the page and explain CFGs and push-down automata using the diagram. Give one example before looking again.

CONCEPT 04 / TOC

# Pumping lemma: use it to disprove regularity

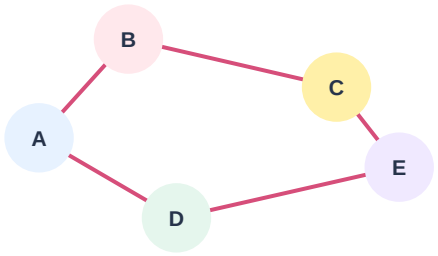
### START WITH THE IDEA

For a regular language, sufficiently long strings can be split xyz with |xy|<=p, |y|>0 and xy^iz in the language for every i>=0. Choose a string that forces y inside a constrained region, then pump.

### A CONCRETE FIRST EXAMPLE

The pumping lemma is a proof tool for showing a language is not regular. It says every sufficiently long string in a regular language can be split into xyz, with a short x+y and repeatable y. For {0■1■}, choose 0■1■. Any legal y lies among the first zeros; pumping it changes the zero count but not the ones.

## PICTURE IT




split a long string into x y z

### IMAGINE IT THIS WAY

A finite-state machine eventually loops; pumping repeats the loop segment.

### HOW TO USE IT ON A PROBLEM

State the quantifiers: for every valid split, choose an i that breaks membership.

### WATCH OUT

The pumping lemma is necessary, not sufficient, for regularity.

### TRY FROM MEMORY

Cover the page and explain Pumping lemma: use it to disprove regularity using the diagram. Give one example before looking again.

CONCEPT 05 / TOC

# Turing machines and reductions

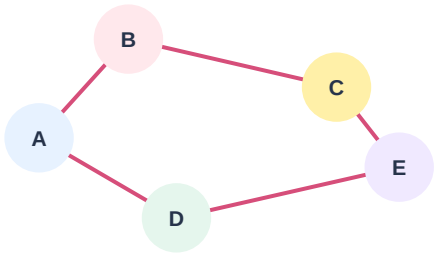
### START WITH THE IDEA

A TM has a read/write tape, head and finite control. Decidable languages have a halting accept/reject procedure. Recognizable languages may loop on nonmembers. Reductions transfer hardness from one problem to another.

### A CONCRETE FIRST EXAMPLE

A Turing machine has a finite control, an unbounded tape and a head that reads, writes and moves. A recognizer may run forever on a no-instance; a decider halts on every input. A tape can hold a working copy of the input while the machine marks symbols already matched.

## PICTURE IT




read / write / move

### IMAGINE IT THIS WAY

A TM is a worker with an erasable infinite notebook and a cursor.

### HOW TO USE IT ON A PROBLEM

Separate acceptance, rejection and nontermination.

### WATCH OUT

A recognizer is not necessarily a decider.

### TRY FROM MEMORY

Cover the page and explain Turing machines and reductions using the diagram. Give one example before looking again.

CONCEPT 06 / TOC

# Undecidability

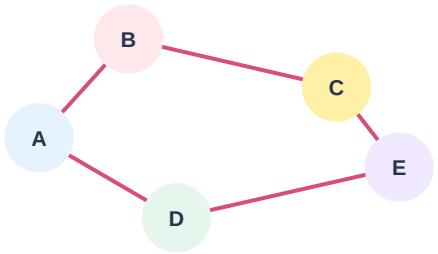
### START WITH THE IDEA

The halting problem is undecidable. A reduction maps instances of a known hard problem to a new problem while preserving yes/no answers. If the new problem were decidable, the known one would be too.

### A CONCRETE FIRST EXAMPLE

Undecidability means no algorithm can always halt and answer correctly for every input. Reductions transfer difficulty: if a known undecidable problem could be solved using your problem, your problem must be at least as hard. To prove a new problem undecidable, assume it has a decider and use it to decide the known Halting Problem; the contradiction rules it out.

## PICTURE IT




halt with yes or no

### IMAGINE IT THIS WAY

A reduction is a translator that would turn a solver for a new puzzle into a solver for an impossible old puzzle.

### HOW TO USE IT ON A PROBLEM

Choose reduction direction carefully: known hard problem -> target.

### WATCH OUT

Do not claim undecidability from intuition alone; show a valid mapping and equivalence.

### TRY FROM MEMORY

Cover the page and explain Undecidability using the diagram. Give one example before looking again.

# Let's solve it together

Cover the answer with your hand. Try each step first, then compare your reasoning.

## EXAMPLE 1: Pump a nonregular language

**QUESTION**

Show L={a^n b^n : n>=0} is not regular.

**WORK THROUGH THE STEPS**

Assume pumping length p. Choose s=a^p b^p. Any split xyz with |xy|<=p and |y|>0 has y=a^k, k>0. Pump i=0: a^(p-k)b^p, which is not in L.

**ANSWER** L is not regular.

Check: write the key rule beside your answer. If it does not fit the rule, redo the model before recalculating.

## EXAMPLE 2: NFA subset step

**QUESTION**

An NFA state set after reading prefix w is {q0,q2}. On symbol 1, q0 goes to q1 and q2 goes to q2; epsilon-closure adds q3 from q1. What is the DFA next subset?

**WORK THROUGH THE STEPS**

**ANSWER** {q1,q2,q3}.

Check: write the key rule beside your answer. If it does not fit the rule, redo the model before recalculating.

# Now you try

Attempt these without the notes. The question types are marked MCQ, MSQ or NAT. Answers and explanations are on the next page.

**MCQ** Which language is regular?

(A) {a^n b^n} (B) {ww : w in {0,1}*} (C) Strings over {0,1} ending in 01 (D) {a^p : p is prime}

**MCQ** A DFA and an NFA recognize:

(A) Different language classes (B) Exactly the regular languages (C) Only finite languages (D) Only context-free languages

**MSQ** Which are true?

(A) Every regular language is context-free (B) Every CFL is regular (C) A PDA has a stack (D) A recognizer must halt on nonmembers

**MCQ** For a pumping-lemma proof of nonregularity, the string s must satisfy:

(A) |s|<p (B) |s|>=p (C) s is arbitrary (D) s is a DFA state

**MCQ** The halting problem asks whether:

(A) A DFA has a final state (B) A program halts on an input (C) A CFG is ambiguous (D) A string is a palindrome

Exam habit: decide what is being asked before doing arithmetic. Show units, assumptions and one line of reasoning.

# Answers with the why

Read the explanation even when your answer is correct. The goal is to learn the reasoning pattern.

**1. C**

**WHY**

A DFA can track the last two symbols; the other listed languages are nonregular.

**2. B**

**WHY**

Subset construction converts any NFA to an equivalent DFA.

**3. A, C**

**WHY**

CFLs strictly include regular languages; recognizers may loop on nonmembers.

**4. B**

**WHY**

The pumping lemma applies to strings at least as long as the pumping length.

**5. B**

**WHY**

No algorithm decides halting for every program-input pair.

Review loop: mark each missed concept page, redraw its picture, and retry a similar problem tomorrow.

# Theory of Computation



**Published By:**

**Physics Wallah**

**ISBN:** 978-93-94342-39-2

**Mobile App:** Physics Wallah (Available on Play Store)

**Website:** www.pw.live

**Email:** support@pw.live

## Rights

All rights will be reserved by Publisher. No part of this book may be used or reproduced in any manner whatsoever without the written permission from author or publisher.

In the interest of student's community:

Circulation of soft copy of Book(s) in PDF or other equivalent format(s) through any social media channels, emails, etc. or any other channels through mobiles, laptops or desktop is a criminal offence. Anybody circulating, downloading, storing, soft copy of the book on his device(s) is in breach of Copyright Act. Further Photocopying of this book or any of its material is also illegal. Do not download or forward in case you come across any such soft copy material.

## Disclaimer

A team of PW experts and faculties with an understanding of the subject has worked hard for the books.

While the author and publisher have used their best efforts in preparing these books. The content has been checked for accuracy. As the book is intended for educational purposes, the author shall not be responsible for any errors contained in the book.

The publication is designed to provide accurate and authoritative information with regard to the subject matter covered.

This book and the individual contribution contained in it are protected under copyright by the publisher.

***(This Module shall only be Used for Educational Purpose.)***

# Theory of Computation

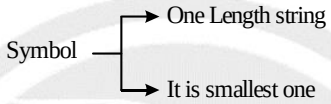
## INDEX


1. Basics of Theory of Computation ..... 7.1 – 7.5
2. Finite Automata ..... 7.6 – 7.21
3. Push Down Automata ..... 7.22 – 7.29
4. Turning Machine ..... 7.30 – 7.39

# 1 BASICS OF THEORY OF COMPUTATION

## 1.1 Symbol

Symbol represents very unique in the world. Any small thing that never be broken into any other is called as symbol.




## 1.2 Alphabet (∑)

It is a set of finite number of symbols.

**Example:**

- English alphabet = {a, b, ... z}
- Binary alphabet = {0, 1}
- Decimal alphabet = {0, 1, 2, ... 9}
- We can create our own alphabet = {gate, cs, it, exam}
  where gate, cs, it, exam all are symbols

Alphabet (Σ) = {gate, cs, it, exam}
{ w , x , y , z }

## 1.3 String

- It is sequence of symbols defined over given alphabet.
  let ∑ = {a, b}
- Strings possible are ∈, a, b, aa, bb……
- Strings over English Alphabet: deva, gate, exam, etc.
- Strings over Binary Alphabet: 0010, 1011, 1101, 1111, etc.
- Strings over Decimal Alphabet: 3012, 2345, 5438, etc.

Note:

- Different length strings over given alphabet
  I Zero length string → Empty string / Null string ∈ or λ is used to denote empty string length of empty string. |∈| = 0.
  II One length string over ∑ = {a, b} are a, and b (2 strings).
  III Two length strings over ∑ = {a, b} are da, ab, ba, and bb (4 strings).

## 1.4 Operations on Strings

There are various operations on strings:

- Unary and Binary operations

1. **length of a string:** Length of a string is denoted as |w| and is defined as the number of positions for the symbol in the string.

   **Example:** w = aba
   |w| =|aba| = 3

2. **Reversal of a string:** It will reverse or changes the order of a given string w.

   **Example:** w = abb
   $w^R$ = bba

### 1.4.1 Concatenation of Two Strings

Given two strings $w_1$ and $w_2$, we define the concatenation of $w_1$ and $w_2$ to be the string as $w_1w_2$.

**Example:**

$w_1$ = a, and $w_2$= ba

Then $w_1w_2$ = a.ba = aba.

### 1.4.2 Prefix of a String

A substring with the sequence of beginning symbols of a given string is called a "prefix".

**Example:**

(i) w = aaaa

Prefixes = {∈, a, aa, aaa, aaaa}

(ii) w = abcd

Prefixes = {∈, a, ab, abc, abcd}

**Note:**

If |w| = n then $|w^{Rev}|$ = n.

If length of $w_1$ is $n_1$ and length of $w_2$ is $n_2$ then length of $w_1w_2$ = $|w_1 w_2|$ = $n_1 + n_2$.

- Let w be n length string.

(i) Number of prefixes in w = n+1

(ii) Number of non-empty prefixes of w = n

(iii) Number of different length prefixes of w = n + 1

(iv) Number of different length prefixes of w excluding zero length = n

### 1.4.3 Suffix of a String

A substring with the sequence of ending symbols of a given string is called a "suffix".

**Example:**

(i) w = aaaa

Suffixes = {∈, a, aa, aaa, aaaa}

(ii) w = abcd

Suffixes = {∈, d, cd, bcd, abcd}

**Note :**

- Let w be n length string.
  (i) Number of suffixes of w = n + 1
  (ii) Number of non-empty suffixes of w = n
  (iii) Number of different length suffixes of w = n + 1
  (iv) Number of different length suffixes of w excluding zero length = n

### 1.4.4 Substring of a String

- Let w be n length string.

(i) Number of substrings of w
- minimum = n +1 (over 1 symbol)
- maximum = 1+ n + (n – 1) + ....+1 (all characters distinct) =1 + ∑n = ∑n +1

(ii) Number of non-empty substrings of w
- minimum = n
- maximum = ∑n

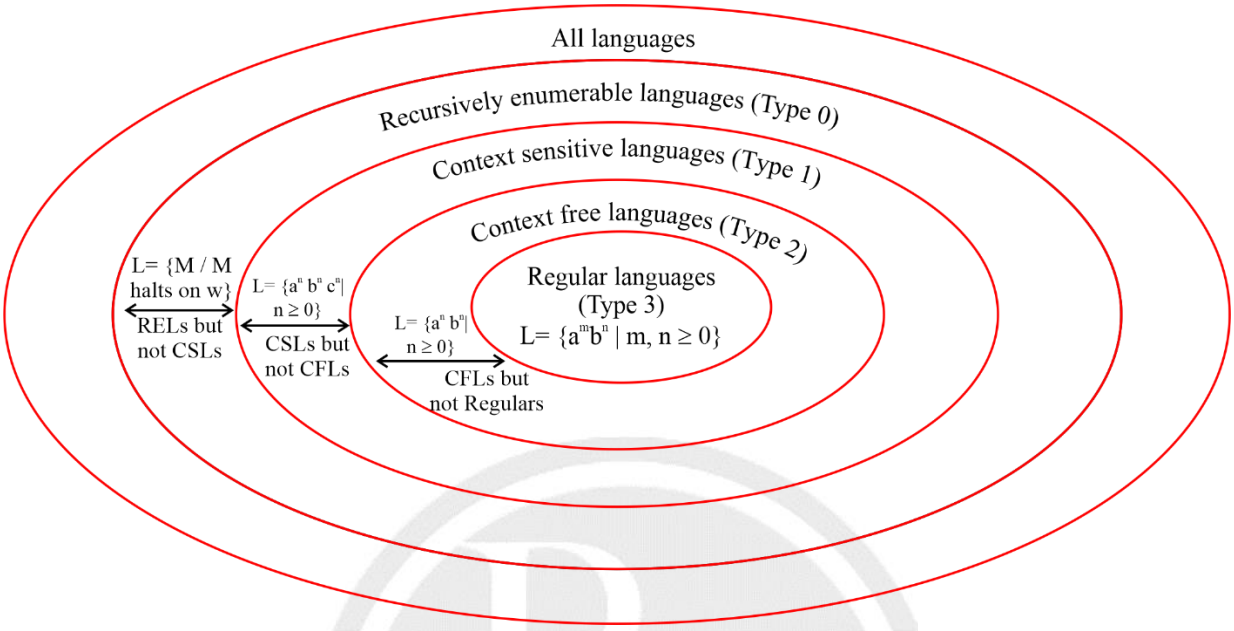
- Number of different length substrings of any given n length string =n+1
- Number of different length substrings of any given n length string excluding zero length=n.

## 1.5 Relation between Symbol, Alphabet, String and Language

| | |
|---|---|
| Character | Symbol |
| ⇓ | ⇓ |
| Character set | Alphabet |
| ⇓ | ⇓ |
| Token | String |
| ⇓ | ⇓ |
| C Language | Language |

### 1.5.1 Chomsky Hierarchy

- Chomsky hierarchy includes all the problems in the world classified into classes.




- Type 3 is the smallest class, and Type 0 is the biggest class.

| | Type 3 | Type 2 | Type 1 | Type 0 |
|---|---|---|---|---|
| **Language:** | Regular | Context free | Context sensitive | Recursively Enumerable |
| **Automata:** | Finite Automata | Push down Automata | Linear Bounded Automata | Turing Machine |
| **Grammar:** | Regular | Context free | Context Sensitive | Unrestricted |

## 1.6 Language

- Language is set of strings defined over alphabet ($\Sigma$).
- Let $\Sigma = \{a,b\}$. Then $\Sigma = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup$ …….
  = set of all strings
  = universal language
  = $\{\in, a, b, aa, ab, ba, ...\}$
- $\Sigma^2 = \Sigma^1 \bullet \Sigma^1 = \{a,b\}.\{a,b\} = \{aa, ab, ba, bb\}$
- Language: It is a subset of $\Sigma^*$

$\therefore$ $\boxed{L \subseteq \Sigma^*}$

- A language is a collection of strings that must be a subset of $\Sigma^*$ where $\Sigma^*$ is a universal language.

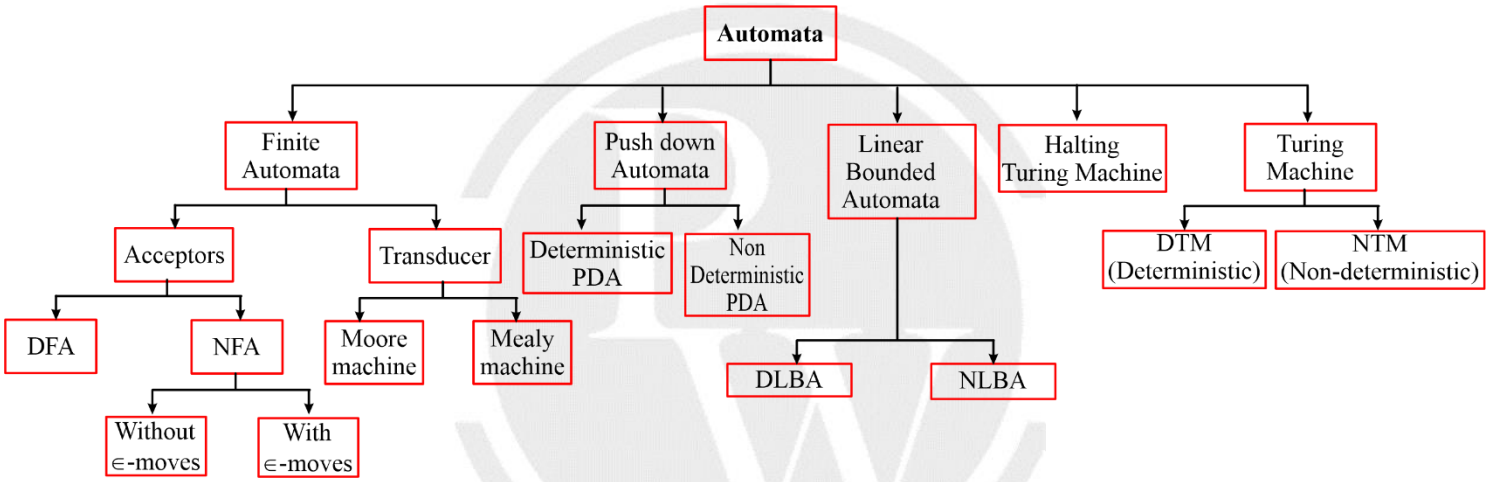
**Example:**

L = ab* = {a, ab, abb,.....}

## 1.7 Types of Languages

- Finite Language
- Infinite Language
- Regular language
- DCFL (Deterministic CFL)
- CFL
- CSL
- Recursive Language
- Recursive Enumerable Language (REL)

## 1.8 Types of automata




- Ralations between these machines:

$$FA < DPDA < PDA < LBA < HTM < TM$$

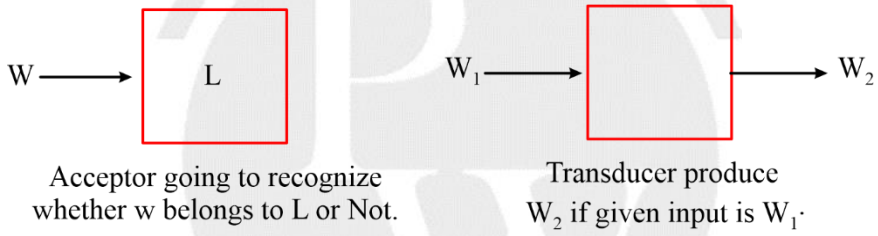
1. **Less power:** It can Represent less number of languages.
2. **More power:** It can Represent more number of languages Compare to all these machines.

❐❐❐

# 2 FINITE AUTOMATA

## 2.1 Introduction

- Finite Automata describes or represents a regular language.
- Finite Automata is of two types:
  (i) Acceptor: Accepts or rejects given string.
  (ii) Transducer: Produces output string for given input string.




Acceptor going to recognize whether w belongs to L or Not.

Transducer produce $W_2$ if given input is $W_1$.

**Example:**

1's complement of binary number: Input = 10100, Output = 01011

## 2.2 Finite Acceptor (Finite Automata)

- Finite Automata:

$FA = (Q, \Sigma, \delta, q_0, F)$

- $F$: Set of final states
- $q_0$: Initial state
- $\delta$: Transition function
- $\Sigma$: Set of input symbols
- $Q$: Set of finite number of states

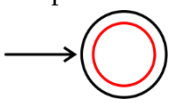
**Transition function (δ)**

(1) DFA : $\delta: Q \times \Sigma, \rightarrow Q$

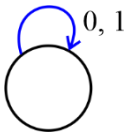
(2) NFA:
- Without $\in-$ moves: $\delta: Q \times \Sigma, \rightarrow 2^Q$
- With $\in-$ moves: $\delta: Q \times \Sigma \cup \{\in\} \rightarrow 2^Q$

## 2.3 Construction of DFA

1. If epsilon belongs to L, then initial state must be final in DFA.

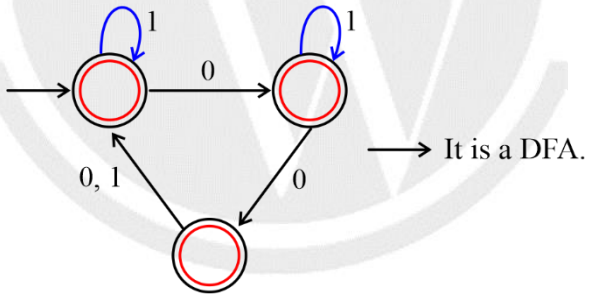


2. **Dead state:** It is non-final state but it never contain a path to final.

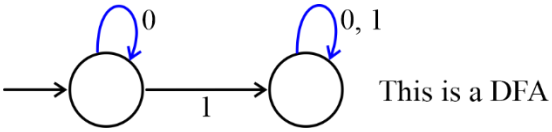



- Once we reach the dead state, there is no way to reach to the final state.
- If every state is final then every string accepted in the DFA.

3. If all states are finals in DFA then L(DFA) = $\sum^*$




4. If every state is non final in a DFA then L(DFA) = $\phi$ or L(DFA) = {},




$\boxed{\overline{L} = \sum^* - L}$

$\boxed{L \cup \overline{L} = \sum^*}$

$\boxed{L \cap \overline{L} = \phi}$

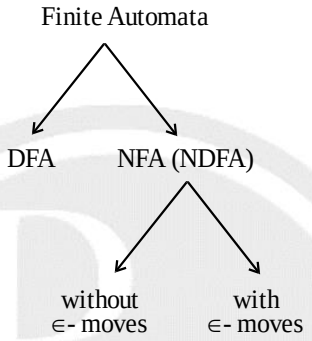
- If $|w| = n$ or $|w| \leq n$, then number of states in minimum DFA = (n + 2) states
- If $|w| \geq n$ then number of states in minimum DFA = n+ 1 states.
- Start condition, exactly, atmost length question requires Dead state but end condition, contain substring, atleast length questions do not require dead state.

- Over one symbol $\sum = \{a\}$. Length of string is equal to number of a's in string. $|w| = n_a(w)$.
- Let $L=\{w|w \in \{a,b\}^*, n_a(w)$ is divisible by m, $n_b(w)$ is divisible by n$\}$. Then Number of states in DFA= m×n.
- Let L={w | w belongs to {a. b}*, $n^{th}$ symbol of w from begin is 'a'}. Then Number of states in DFA = (n + 2).
- Number of equivalence classes for any regular set L = Number of states in minimal DFA that accepts L.

## 2.4 Non-Deterministic Finite Automata

### 2.4.1 Introduction

- Finite Automata can be designed in two ways:




- All these finite state machines are equivalent. DFA ≅ . NFA
- We can convert one finite state machine to any other finite state machine.
- For every gular language, we can design infinite equivaent DFAs or infinite equivalent NFAs but minimum DFA is uique for given regular language.
- For every regular language, one or more minimum NFAs may exist.

**Note :** For every regular language:

(i) Unique minimum DFA exists.

(ii) One or more minimum NFAs exists.

## 2.5 Comparison of NFA and DFA

| | **$\|w\| = n$** | **$\|w\| \le n$** | **$\|w\| \ge n$** |
|---|---|---|---|
| **Number of states** in NFA | n + 1 | n + 1 | n + 1 |
| **Number of states** in DFA | n + 2 | n + 2 | n + 1 |

- If every string has $n^{th}$ symbol from begin is 'a' over binary alphabet {a, b} then (n + 1) states in minimum NFA but (n+2) states in minimum DFA.

- If every string has $n^{th}$ symbol from end is 'a' over binary alphabet {a, b} then $2^n$ states in minimum DFA and (n + 1) states in minimum NFA.

## 2.5.1 COMPARISON OF DFA AND NFA (NFA vs DFA)

| | NFA | DFA |
|---|---|---|
| (1) Transition Function ($\delta$) | $Q \times \Sigma \rightarrow 2^Q$ | $Q \times \Sigma \rightarrow Q$ |
| (2) Number of paths for string | For valid string: 1 path<br>For invalid string: >=0 paths | For valid string: 1 path<br>For invalid string: 1 path |

## 2.5.2 Number of states in DFA and NFA for Regular Languages

| Language | NFA states | DFA states |
|---|---|---|
| (1) {w \| w ∈ {a, b}*, \|w\| = n} | n + 1 | n + 2 |
| (2) {w \| w ∈ {a, b}*, \|w\| ≤ n} | n + 1 | n + 2 |
| (3) {w \| w ∈ {a, b}*, \|w\| ≥ n} | n + 1 | n + 2 |
| (4) {w \|w ∈ {a, b}*, w starts with a} | 2 | 3 |
| (5) {w\| w∈ {a, b}*, $n^{th}$ symbol from begin is a} | n + 1 | n + 2 |
| (6) {w\| w ∈ {a, b}*, $n^{th}$ symbol from end is 'a'} | n + 1 | $2^n$ |

## 2.5.3 Finding number of states in DFA using NFA

**NFA (n states)**

⇓ Subset Construction

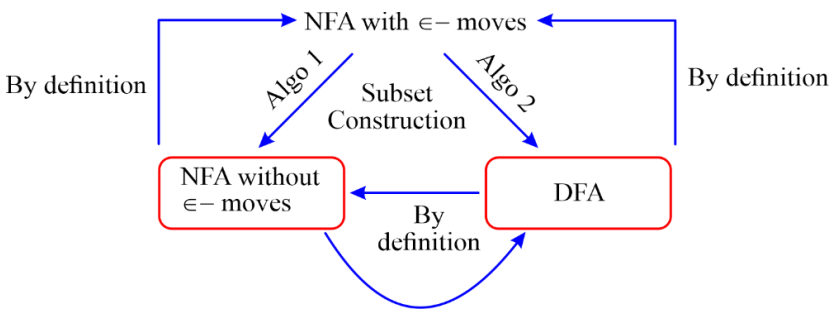
**DFA ($2^n$ sates exists)**

⇓ Partition algorithm

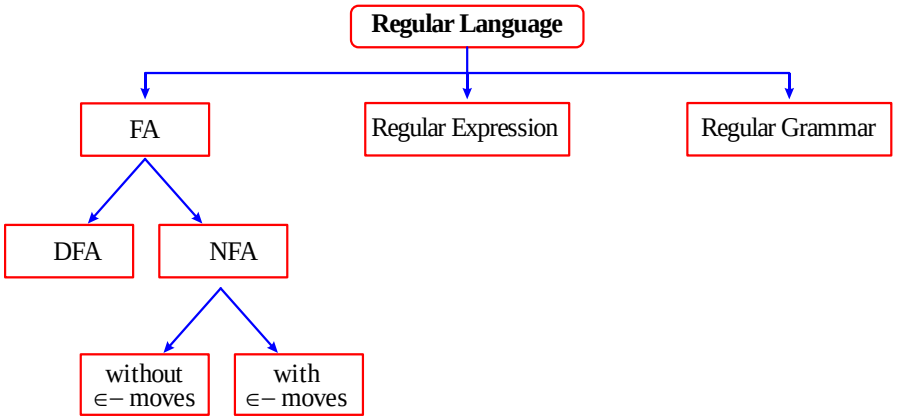
**Minimum DFA (≤ $2^n$ states) atmost $2^n$ states**

- Every DFA is NFA, but NFA need not be DFA.

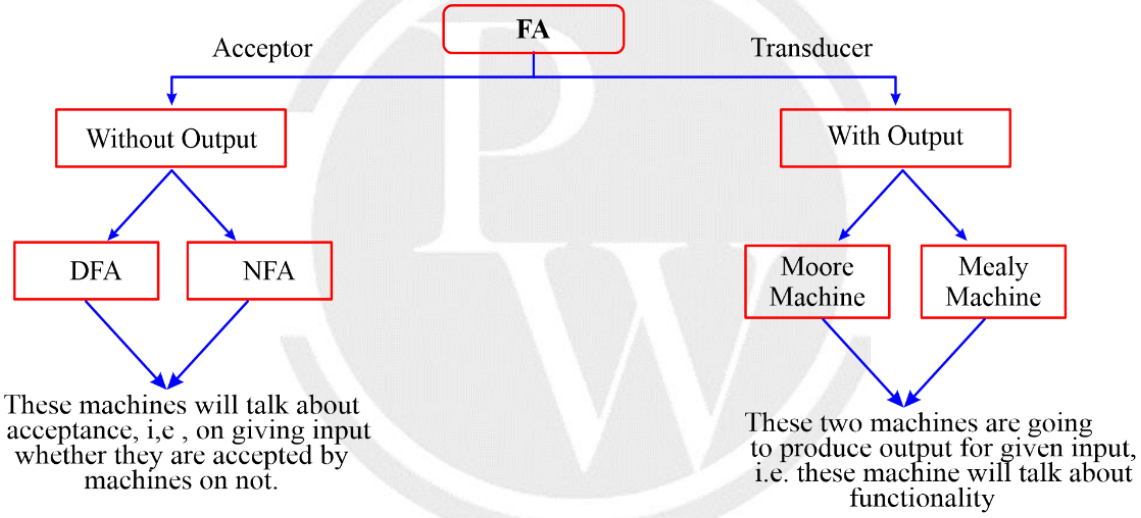
## 2.5.4 Relation between NFA with epsilon, NFA without epsilon, and DFA



## 2.6 Regular language Representation




## 2.7 FA Classification

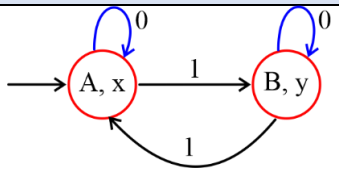



## 2.8 Moore machine and mealy machine

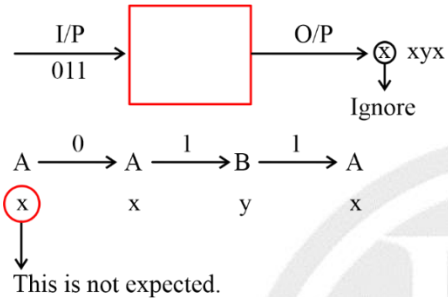
| Moore machine | Mealy machine |
|---|---|
| Transition Function $\delta : Q \times \Sigma \rightarrow Q$<br>Output Function $\lambda : Q \rightarrow \Delta$<br>Outpput is asociated with every state. | $\delta : Q \times \Sigma \rightarrow Q$<br>$\lambda : Q \times \Sigma \rightarrow \Delta$<br>Output is associated with transition. |

### 2.8.1 Example for Moore machine and Mealy machine

**Moore**

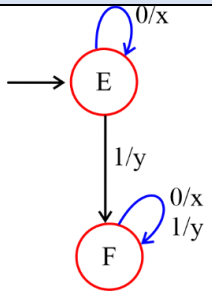



Q = {A, B}
∑ = {0, 1}
Δ = {x, y}
Outpput is asociated with state.

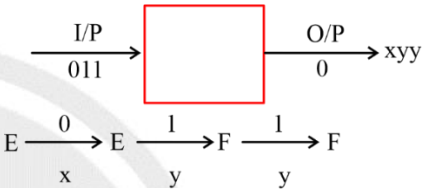



- Extra one output is produced other than desired output.
- So, we can ignore this extra output as this is the machine property.

**Mealy**




Q = {A, B}
∑ = {0, 1}
Δ = {x, y}
Output is associated with transition.

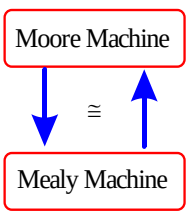



- No extra output is produced.
- For 3 length input, we are getting the 3 length output.

## 2.9 Difference between Moore machine and Mealy machine

| | | Moore machine | Mealy machine |
|---|---|---|---|
| 1. | δ | $Q \times \Sigma \to Q$ | $Q \times \Sigma \to Q$ |
| 2. | λ | $Q \to \Delta$ | $Q \times \Sigma \to \Delta$ |
| 3. | Length of O/p | If n length I/P (assume 1 length O/P symbol is taken at each state) then O/P length is (n+1). | If n length input (assume 1 length O/P symbol is taken at each transition) is given then O/P length is n. |
| 4. | By default | DFA no final state. | DFA no final state. |

## 2.10 Constuction of FA with output



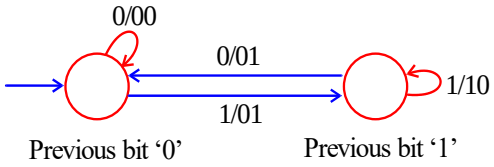
**Note :**

For every problem, if moore machine exists the we can also construct equivalent mealy machine.

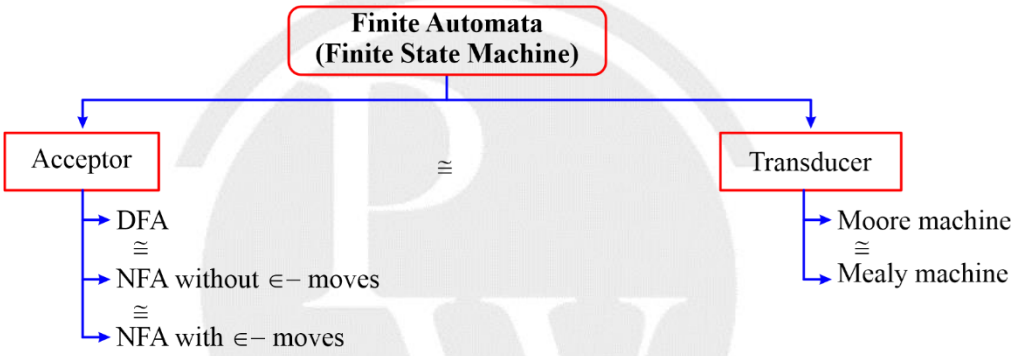
**Example:**

Sum of present bit and previous bit.

Mealy Machine



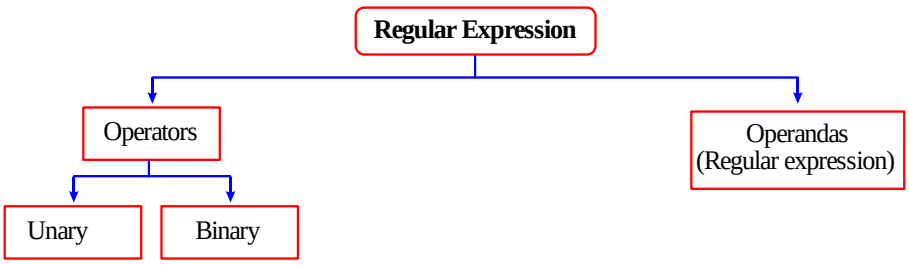

## 2.11 Classification of Finite State Machine (FSM)




## 2.12 Regular Expression

**Definition:**

- Regular expression represents a regular language.
- It describes a regular set.
- L (regular expression) is regular set.
- It is a kind of declarative way to represents a regular language.
- Regular expression generates a regular set.
- It uses 4 operators to represent a regular language.



## 2.13 Operators of Regular Expression

1. OR (+)
2. Concatenation(.)

} Binary Operator

3. Kleenestar (∗)
4. Kleenestar (+)

} Unary Operator

| Regular Expression | Equivalent Regular Set |
|---|---|
| a + b | L(a + b) = {a, b} |
| a + a = a | L(a + a) = {a} |
| a + ϕ = a | L(a + ϕ) = {a} |
| a + ∈ = ∈ + a | L(a + ∈) = {a, ∈} |
| ∈ + ∈ = ∈ | L(∈+∈) = {∈} |
| ϕ + ϕ = ϕ | L(ϕ + ϕ) = {} |
| a · b = ab | L (a · b) = {ab} |
| a · ∈ = a | L (a · ∈) = {a} |
| ∈ · ∈ = ∈ | L(∈·∈) = {∈} |
| ϕ · ϕ = ϕ | L(ϕ · ϕ) = {ϕ} |
| a · a = aa = $a^2$ | L (a · a) = aa = {$a^2$} |

## 2.14 Kleene star/ kleene closure / closure

**R* ( Kleene closure of R):**

$$R^* = R^\circ + R^1 + R^2 + R^3 + \ldots = \in + R + RR + RRR + \ldots.$$

**Example:**

$a^* = \{a^0, a^1, a^2, a^3, \ldots\} = \{\in, a, aa, aaa, \ldots\}$ = Set of all strings over a.

$\phi^* = \phi^0 + \phi^1 + \phi^2 + \phi^3 + \ldots. = \in + \phi + \phi + \phi + \ldots. = \in + \phi = \in$

## 2.15 Positive Closure (Kleeme Plus)

**$R^+$ (Positive Closure of R):**

$R^+ = R^1 + R^2 + R^3 + \ldots..$

$R^* = R^+ + R^0$

$R^* = R^{\geq 0}$ i.e. repeart R any number of time

$R^+ = R^{\geq 1}$ i.e. repeat R atleast 1 time.

**Example:**

(1) $a^+ = \{a, aa, aaa, \ldots\} = \{a^n \mid n \geq 1\}$

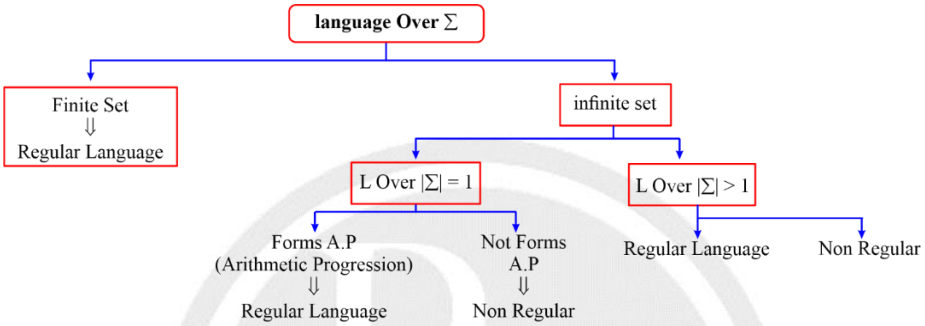
(2) $\in^+ = \in = \in^*$

(3) $\phi^+ = \phi$

**Properties :**

| | | OR | Concatenation |
|---|---|---|---|
| 1. | Identity | $\phi$ | $\in$ |
| 2. | Associative | Yes | Yes |
| 3. | Commutative | Yes | No |
| 4. | Annihilatior | $\Sigma^*$ | $\phi$ |

## 2.16 Identification of Regular Languages




1. $L=\{a^nb^m \mid n, m \geq 0\} = a^*b^*$ (Regular language)
2. $L= \{a^nb^m \mid m<n<10\}$ (Finite Set, Regular language)
3. $L=\{a^nb^m \mid m > n > 10\}$ (Non Regular language)
4. $L= \{a^nb^m \mid m = n, m < 10\}$ (Regular language)
5. $L=\{ a^nb^m \mid m = n, m > 10\}$ (Non Regular language)
6. $L=\{a^mb^n \mid \gcd(m, n) = 1\}$ (Non Regular language)
7. $L=\{a^mb^n \mid LCM(m, n) = 1\}$ (Regular language)
8. $L= \{a^nb^n \mid n >= 0\}$ (Non Regular language)
9. $L=\{a^nb^{2n} \mid n \geq 0\}$ (Non Regular language)
10. $L=\{a^mb^n \mid m=\text{even}, n=\text{odd}\}$ (Regular language)
11. $L=\{a^mb^n \mid m = n = \text{even}\}$ (Non Regular language)

    OR

    $L = \{a^{2n}b^{2n} \mid n = \text{even}\}$
12. $L=\{a^*\}$ over $\Sigma= \{a\}$

    = Regular language
13. $L=\{a^{2n} \mid n >= 0\}$ over $\Sigma = \{a\}$.

    $L= a^{2n}=(aa)^*$

    = Regular language

14. L= $\{a^{Prime}\}$ over $\sum = \{a\}$

    = Non regular language

15. L=$\{ a^{n^2} \mid n \geq 0\}$ over $\sum = \{a\}$

    L={epsilon, a, aaaa, $a^9$, $a^{16}$...}, FA Not possible for L. So, it's Non-Regular language.

16. L= $\{ a^{2^n} \mid n > 0\}$over $\sum = \{a\}$

    Non Regular language

17. L= $\{ a^{2^n} \mid n \leq 10\}$

    = Regular language

18. L=$\{a^{n!} \mid n \geq 100\}$ over S = {a}

    = Non Regular language

19. L=$\{ a^{n^n} \mid n \geq 10\}$ over $\sum = \{a\}$

    = Non Regular language

20. L = $\{a^{Prime}\}^*$ over $\sum = \{a\}$

    L = complement of {a} = $\sum^* - \{a\} = \{\in, a^2, a^3\ a^4, a^5, a^6, a^7, ...\}$= Regular language

21. L = $\{a^{prime} \mid$ prime <100$\}$ is finite language (regular)

22. L = $\{w \# w \mid w \in a^*\}$

    L = $\{a^n \# a^n\}$

    dependency

    = Non Regular language

23. L = $\{w \# w \mid w \in (a+b)^*\}$

    = Non Regular language

24. L = $\{w \# w^R \mid w \in a^*\}$ is non regular

    L = $\{a^n \# a^n\}$

25. L = $\{w\ x\ w \mid w \in \{a, b\}^*, x \in \{a, b\}^+\}$

    Put w = $\in$, and x = $(a + b)^+$

    L = $(a + b)^+$

    = Regular language

26. L = $\{x\ w\ w \mid w \in \{a, b\}^*\ x \in \{a, b\}^+\}$

$L = (a + b)^+$

= Regular language

27. $L = \{w\ w^R\ x \mid w \in \{a, b\}^*, x \in \{a, b\}^+\}$

$L = (a + b)^+$

= Regular language

28. $L = \{w\ x\ w^R \mid w \in (a + b)^*, x \in \{a, b\}^+\}$

$L = (a + b)^+$

= Regular language

29. $L = \{x\ w\ w^R \mid w \in \{a, b\}^*, x \in \{a, b\}^+\}$

$L = (a + b)^+$

## 2.17 Closure Properties of Regular Languages

**1. Closure properties for finite languages:**

- Union ✓
- Intersection ✓
- Complement ×
- Difference ✓
- Concatenation ✓
- Reversal ✓
- Kleene closure ×
- Subset ✓

## 2.18 Table for FINITE sets

| | | Finite sets | Closed/Not Closed |
|---|---|---|---|
| (1) | Union ($\cup$) | $F_1 \cup F_2 \Rightarrow$ Finite | ✓ |
| (2) | Intersection ($\cap$) | $F_1 \cap F_2 \Rightarrow$ Finite | ✓ |
| (3) | Complement ($\overline{L}$) | $\overline{F} \Rightarrow$ NOT finite | × |
| (4) | $L_1 - L_2$ | $F_1 - F_2 \Rightarrow$ Finite | ✓ |
| (5) | $L_1 . L_2$ | $F_1 \cdot F_2 \Rightarrow$ Finite | ✓ |
| (6) | $L^*$, $L^+$ | $F^*$, $F^+ \Rightarrow$ May or may not be finite | × |
| (7) | Subset (L) | Subset (Finite set) $\Rightarrow$ Finite | ✓ |

### 2.18.1 Table for Infinite sets

| | | Infinite sets |
|---|---|---|
| (1) | Union ($\cup$) | Infinite $\cup$ Infinite $\Rightarrow$ Infinite |
| (2) | Intersection ($\cap$) | Infinite $\cap$ Infinite $\Rightarrow$ Need not be infinite |
| (3) | Complement ($\overline{L}$) | Complement of Infinite $\Rightarrow$ May or may not be infinite |
| (4) | $L_1 - L_2$ | Need not be infinite |
| (5) | $L_1 . L_2$ | Infinite |
| (6) | $L^*, L^+$ | Infinite |
| (7) | Subset (L) | Need not be infinite |

## 2.19 Closure Properties of Regulars

$L_i \rightarrow$ Regular

| | | | |
|---|---|---|---|
| 1. | $\cup$ | 2. | $\cap$ |
| 3. | $\overline{L}$ | 4. | $L_1 - L_2$ |
| 5. | $L_1 \cdot L_2$ | 6. | $L^{Rev}$ |
| 7. | $L^+$ | 8. | $L^+$ |
| 9. | Subset (L) is not closed for regular languages | 10. | Prefix (L) |
| 11. | Suffix (L) | 12. | Substring (L) |
| | | | |
| 13. | Substitution (L) | 14. | Homomorphism (L) |
| 15. | $\in$-free homomorphism (L) | 16. | $h^{-1}$ (L) (Inverse homomorphism (L)) |
| 17. | $L_1/L_2$ (Quotient) | 18. | Symmetric difference |
| 19. | Half (L) | 20. | Second half (L) |
| 21. | One-third (L) [1/3 (L)] | 22. | Middle 1/3 (L) |
| 23. | Last 1/3 (L) | 24. | New $(L_1, L_2) = \text{Suffix}(L_1 \cup L_2^+)$ |
| 25. | Finite Union | 26. | Finite Intersection |
| 27. | Finite Difference | 28. | Finite Concatenation |

| | | | |
|---|---|---|---|
| 29. | Finite Subset | 30. | Finite Substitution |
| 31. | Infinite Union | 32. | Infinite $\cap$ |
| 33. | Infinite Difference | 34. | Infinite Concatenation |
| 35. | Infinite Subset | 36. | Infinite Substitution |

- **Out of the given 36 closure properties, how many are closed for regular languages?**

  Subset operation, and 6 infinite operations are not closed, remaining all are closed for regular languages.

  Out of 36 operations total 7 operations are not closed.

## 2.19.1 Operations over regular languages

**Examples:**

(1) $\left.\begin{matrix} L_1 = a^* \\ L_2 = a^+ \end{matrix}\right\} \Rightarrow L_1 \,.\, L_2 = a^*$

(2) $\left.\begin{matrix} L_1 = \phi \\ L_2 = \text{Any} \end{matrix}\right\} \Rightarrow L_1 + L_2 = L_2$

(3) $\left.\begin{matrix} L_1 = a^* \\ L_2 = b^* \end{matrix}\right\} \Rightarrow L_1 + L_2 = a^* + b^*$

(4) $\left.\begin{matrix} L_1 = \Sigma^* \\ L_2 = \text{Any language over same } \Sigma \end{matrix}\right\} \quad L_1 + L_2 = \Sigma^* = L_1$

(5) $\left.\begin{matrix} L_1 = a\Sigma^* \\ L_2 = b\Sigma^* \end{matrix} \quad \Sigma = \{a,b\}\right\} \quad \begin{aligned} L_1 + L_2 &= a\Sigma^* + b\Sigma^* \\ &= (a+b)\Sigma^* \\ &= \Sigma^+ \end{aligned}$

**Note :**

1. $\left.\begin{matrix} L_1 = \phi \\ L_2 = \text{Any} \end{matrix}\right\} \Rightarrow L_1 \cdot L_2 = \phi$

2. $\left.\begin{matrix} L_1 = \Sigma^* \\ L_2 = \text{Any} \end{matrix}\right\} \Rightarrow L_1 + L_2 = L_2$

## 2.19.2 Properties of regular languages

I. If both $L_1$ and $L_2$ are regular sets then $L_1 \cap L_2$ is Regular.

II. IF $L_1 \cap L_2$ is regular set then

- $L_1$ "need not be regular"

- $L_2$ "need not be regular".

III. If L is regular then $\overline{L}$ is regular.

IV. If $\overline{L}$ is regular then L is regular.

V. L is regular iff $\overline{L}$ is regular

VI. L is not regular iff $\overline{L}$ is not regular.

**Example:**

$a^n b^n$ is not regular and $\overline{a^n b^n}$ is not regular.

### 2.19.3 Arden's Lemma and Kleene Method

**Arden's Lemma:**

If R = Q + RP and P does not contain $\in$ then $\boxed{R = QP^*}$

$R = Q + RP$ ...(1)

$= Q + (Q + RP)\,P = Q + QP + RP^2$ ...(2)

Substitute R one more time in equation (2)

$R = Q + QP + QP^2 + RP^3$

If we do repetitive substitution infinite time, we will get $\boxed{R = QP^*}$

**Kleene Method:**

**Kleene Method**

→ Dynamic programming $O(n^3)$

→ $\boxed{R_{ij}^{k}}$ where *i* is the initial state, *j* is the final state
k represents number of states

$$\boxed{R_{ij}^{k} = R_{ij}^{k-1} + R_{ik}^{k-1}\left(R_{kk}^{k-1}\right)^{*} R_{kj}^{k-1}}$$

**Regular Grammar**

→ It represents a regular set

→ It can be either left linear grammar (LLG) or right linear grammar (RLG)

| LLG | RLG |
|---|---|
| Each production in LLG follows<br>$V \rightarrow VT^*$<br>OR<br>$V \rightarrow T^*$ | Each production in RLG follows<br>$V \rightarrow T^*V$<br>OR<br>$V \rightarrow T^*$ |

## 2.20 Identify the Language Generated by Regular Grammar

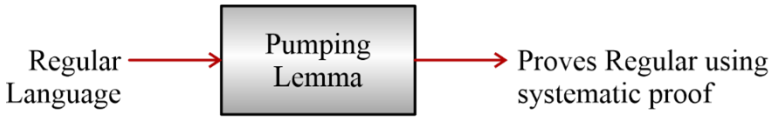
| | Regular Grammar | Regular Language |
|---|---|---|
| 1. | $S \rightarrow \in$ | $L = \{\in\}$ |
| 2. | $S \rightarrow \in \mid a$ | $L = \{\in, a\}$ |
| 3. | $S \rightarrow aa \mid abc \mid d$ | $L = \{aa, abc, d\}$ |

| | | |
|---|---|---|
| 4. | S → Aa<br>A → b | By default S is a start symbol here.<br>L = {ba} |
| 5. | S → Aa | Useless production. It has no meaning.<br>L = { } = ϕ |
| 6. | $S \rightarrow \underset{\text{Useless}}{\boxed{Ab}} \mid b$ | L = {b} |
| 7. | ↑ S → Aa \| Ba<br>A → a<br>B → b<br>Look from bottom to top | L = {aa, bb} |
| 8. | ↑ S → Aa<br>A → ∈\|b | L(A) = {∈, b}<br>L = L(S) = L(A) · a<br>= {a, ba} |

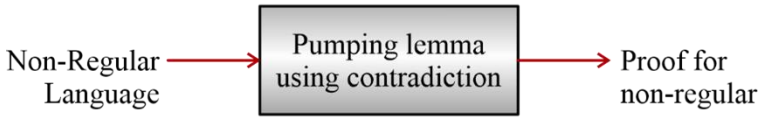
| Regular Language | Regular Grammar |
|---|---|
| $a^*$ | (I) S → Sa \| ∈<br>OR<br>(II) S → aS \| ∈ |
| $a^+$ | (I) S → Sa \| a<br>OR<br>(II) S → aS \| a |
| $(a + b)^*$ | (I) S → Sa \| Sb \| ∈<br>OR<br>(II) S → aS \| bS \| ∈ |
| $(a + b)^+$ | (I) S → Sa \| Sb \| a \| b<br>OR<br>(II) S → aS \| bS \| a \| b |
| $(ab)^*$ | (I) S → Sab \| ∈<br>OR<br>(II) S → abS \| ∈ |

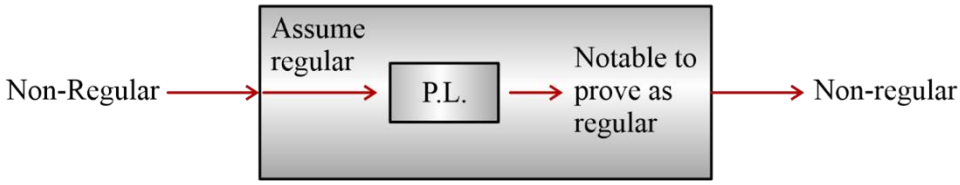
## 2.21 Pumping-Lemma (PL)

### 2.21.1 Pumping Lemma for Regular Languages



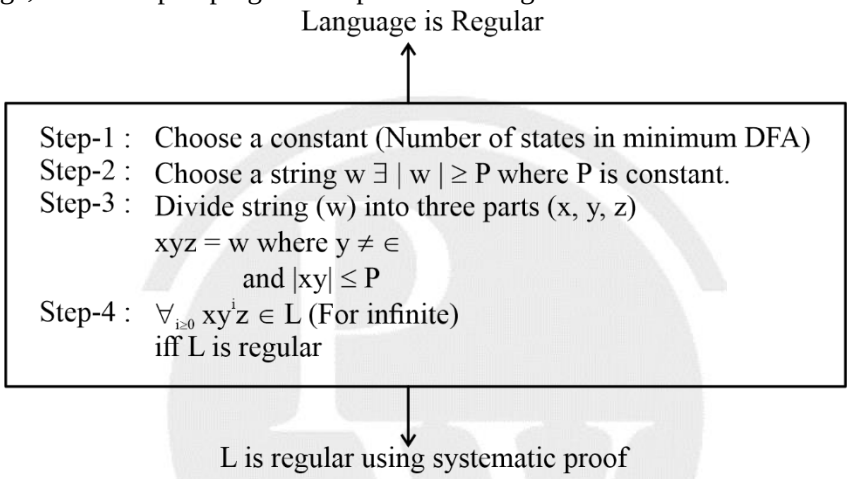

### 2.21.2 Pumping Lemma for Non-Regular Languages using contradiction



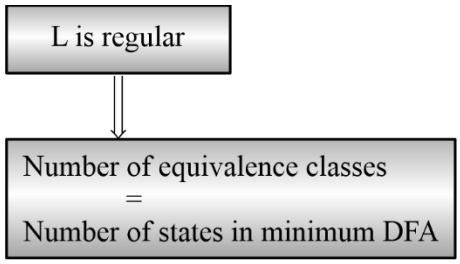

### 2.21.3 Proof for Regular Languages

- If L is Regular language, then how pumping lemma proves it as regular?




## 2.22 Equivalence Classes

(1) L is regular iff L has finite number of equivalence classes.
(2) L is not regular iff L has infinite equivalence classes.

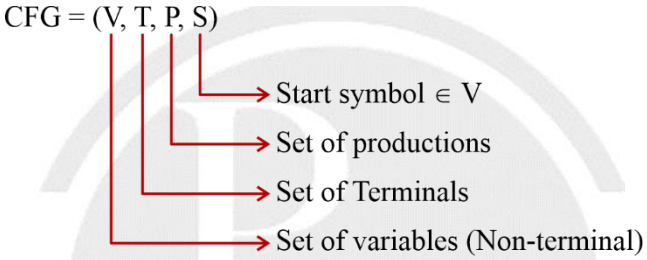



❐❐❐

# 3 PUSH DOWN AUTOMATA

## 3.1 Context Free Grammar

**CFG:** It represents a Context Free Language.




- Rule of each production in P:
  $V \rightarrow (V \cup T)^*$
  V = only 1 variable in LHS
- To derive a string, following derivations can be used.

1. **Linear Derivation:** Linear derivation is two types

   (a) Left Most Derivation (LMD)

   (b) Right Most Derivation (RMD)

2. **Non- linear Derivation:** Non- linear Derivation OR Parse Tree OR Derivation Tree

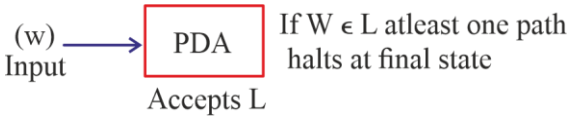
## 3.2 Types of Context Free Grammars

There are two types of CFG:

1. **Ambiguous CFG:** At least one string has more than one derivation.
2. **Unambiguous CFG:** Every string (w) generated by CFG has exactly one derivation.

## 3.3 Pushdown Automata (PDA)

PDA accepts context free language (CFL). PDA also called as NPDA.




## 3.4 PDA acceptance mechanisms

- PDA acceptance mechanisms are three types:
  1. PDA acceptance using final state.
  2. PDA acceptance using empty stack.
  3. PDA acceptance using both final state and Empty stack.
     All PDA acceptance mechanisms are equivalent.
- DPDA acceptance mechanism are two types:
  1. DPDA using final stack.
  2. DPDA using both final state and empty stack.

## 3.5 PDA configuration

PDA = $(Q, \sum, \delta, q_0, F, Z_0, \Gamma)$.

where Q = Set of states

$\sum$ = input alphabet

$\delta$ = Transition Function (PDA/NPDA $\delta : Q \times \sum \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$)

F = Set of Final state

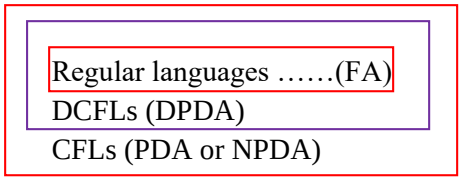
$Z_0$ = Bottom symbol or initially TOS

$\Gamma$ = Stack Alphabet

- DPDA transition Function is [$\delta : Q \times \sum \times \Gamma \rightarrow Q \times \Gamma^*$]
- Difference between DPDA and PDA

| DPDA | PDA |
|---|---|
| [1] $\delta : Q \times \sum \times \Gamma \rightarrow Q \times \Gamma^*$ | $\delta : Q \times \sum \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$ |
| [2] Every DPDA is PDA | Every PDA need not be DPDA |
| [3] DPDA acceptance with<br>(a) Final state mechanism<br>(b) Both Final and empty stack | PDA acceptance with<br>(a) Final state mechanism<br>(b) Empty stack mechanism<br>(c) Both Empty stack and Final state |

- Relation between Regular, DCFL and CFL.



I If L is regular language, then it is also DCFL and CFL.

II Every DCFL is CFL, but it need not be regular.

## 3.6 Closure Properties of CFLs

| Operation | Closed / Not Closed |
|---|---|
| Union ($L_1 \cup L_2$) | ✓ |
| Intersection ($L_1 \cap L_2$) | × |
| Complement ($\overline{L}$) | × |
| Set difference ($L_1$-$L_2$) | × |
| Concatenation ($L_1.L_2$) | ✓ |
| Reversal ($L^{Rev}$) | ✓ |
| Kleene Closure (L) | ✓ |
| Kleene Plus ($L^{\bullet}$) | ✓ |
| Subset (L) | × |
| Prefix (L) | ✓ |
| Suffix (L) | ✓ |
| Substring (L) | ✓ |
| Substitution (L) | ✓ |
| Homomorphism (L) | ✓ |
| $\in$ - free Homomorphism h(L) | ✓ |
| Inverse Homomorphism $h^{-1}$ (L) | ✓ |
| quotient ($L_1$, $L_2$) = $L_1/L_2$ | × |
| Symmetric difference ($L_1$, $L_2$) | × |
| Finite Union | ✓ |
| Finite Intersection | × |
| Finite difference | × |
| Finite concatenation | ✓ |
| Finite subset | ✓ |
| Finite substitution | ✓ |
| Infinite Union | × |
| Infinite intersection | × |
| Infinite difference | × |
| Infinite concatenation | × |
| Infinite Subset | × |
| Infinite Substitution | × |
| Union with Regular (L U Regular) | ✓ |
| L U Regular | ✓ |
| L – Reg | ✓ |
| Reg – L | × |

**Note :**(i) CFL ∩ CFL = Need not be CFL

(ii) CFL ∩ Regular = CFL (Need not be DCFL)

(iii) CFL ∩ DCFL = May or may not be CFL

(iv) CFL ∩ Finite = Finite

(v) CFL ∩ infinite = Need not be CFL

## 3.7 Closure Properties of DCFLs

| Operation | Closed / Not Closed |
|---|---|
| Union ($L_1 \cup L_2$) | × |
| Intersection ($L_1 \cap L_2$) | × |
| Complement ($\overline{L}$) | ✓ |
| Set difference ($L_1$-$L_2$) | × |
| Concatenation ($L_1.L_2$) | × |
| Reversal ($L^{Rev}$) | × |
| Kleene Closure (L) = L* | × |
| Kleene Plus (L) = $L^+$ | × |
| Subset (L) | × |
| Prefix (L) | ✓ |
| Suffix (L) | × |
| Substring (L) | × |
| Substitution (L) | × |
| Homomorphism (L) | × |
| ∈ - free Homomorphism h(L) | × |
| Inverse Homomorphism $h^{-1}$ (L) | ✓ |
| quotient ($L_1$, $L_2$) | × |
| Symmetric difference ($L_1$, $L_2$) | × |
| Finite Union | × |
| Finite Intersection | × |
| Finite difference | × |
| Finite concatenation | × |
| Finite subset | ✓ |
| Finite substitution | × |
| Infinite Union | × |
| Infinite intersection | × |
| Infinite difference | × |
| Infinite concatenation | × |
| Infinite Subset | × |
| Infinite Substitution | × |
| Union with Regular (L U Regular) | ✓ |
| L U Regular | ✓ |
| L – Regular | ✓ |
| Regular – L | ✓ |

**Note :**

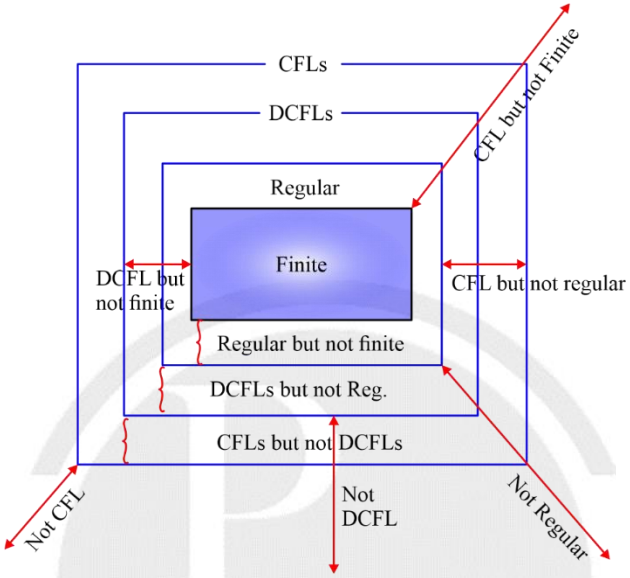
DCFL ∪ Regular : DCFL
DCFL ∩ Regular : DCFL
DCFL - Regular : DCFL
Regular - DCFL : DCFL
DCFL ∪ CFL : CFL (need not be DCFL)
DCFL ∩ CFL : Need not be CFL
DCFL - CFL : Need not be CFL
CFL - DCFL : Need not be CFL
DCFL ∪ Finite : DCFL
DCFL ∩ Finite : Finite
DCFL – Finite : DCFL
Finite - DCFL : Finite

## 3.8 Comparison of Regular Grammars and CFGs

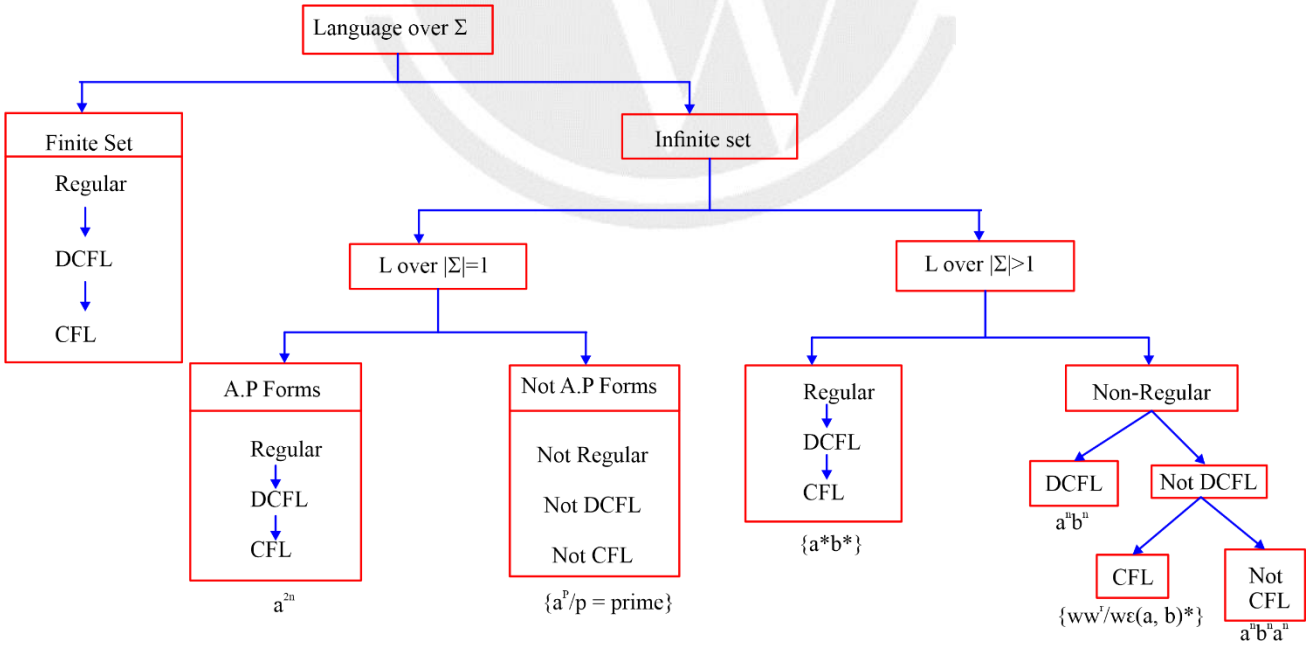
- CFL and CFG both are equivalent. CFL ≅ CFG
- LLG: Left Linear Grammar, RLG: Right Linear Grammar, RG: Regular Grammar, LG: Linear Grammar, CFG: Context Free Grammar.
- Every LLG is RG
- Every RLG is RG
- Every RG is LG
- Every RG is CFG
- Every LG is CFG
- Every RG need not be RLG
- Every RG need not be LLG
- Every LG need not be RG
- Every CFG need not be RLG
- Every CFG need not be LG

## 3.9 Context Free Languages and DCFLs

### I. Comparison of various languages




### II. Identification of Regulars, DCFLs, and CFLs



[1] $L = \{a^n b^n c^k \mid n, m, k \geq 0\}$
$= a^* b^* c^*$
= Regular

[2] $L = \{a^n b^n \mid n \geq 0\}$
L = DCFL but not regular
$S \rightarrow aSb \mid \in$

[3] $L = \{a^n b^{2n} \mid n \geq 0\}$
L = DCFL but not regular
$S \rightarrow aSbb \mid \in$

[4] $L = \{a^{2n} b^n \mid n \geq 0\}$
L = DCFL
$S \rightarrow aaSb \mid \in$

[5] $L = \{a^{2n} b^{2n} \mid n \geq 0\}$
L = DCFL
$S \rightarrow aaSbb \mid \in$

[6] $L = \{ a^n b^n c^* \}$ Assume always $n \geq 0$ in all examples
OR
$= \{ a^m b^n c^* \mid m = n\}$
DCFL but not regular

[7] $L = \{ a^n b^* c^n \}$ DCFL

[8] $L = \{ a^* b^n c^n \}$ DCFL

[9] $L = \{ a^m b^n c^* \mid m \neq n \}$
DCFL

[10] $L = \{ a^m b^n c^* \mid m < n \}$
DCFL

[11] $L = \{ a^m b^n c^* \mid m > n \}$
DCFL

[12] $L = \{ a^m b^n c^* \mid c \leq n \}$
DCFL

[13] $L = \{ a^m b^n c^* \mid m \geq n \}$
DCFL

[14] $L = \{ a^m b^n c^{m+n} \mid m, n \geq 0 \}$ is DCFL

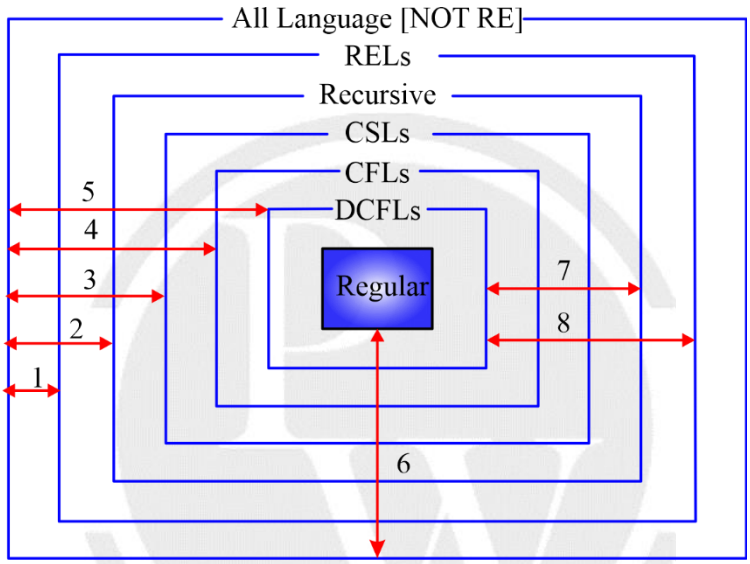
[15] L = { $a^{m+n}b^{n+k}c^{k+m}$ |m, n ≥ 0 } is CFL

[16] L = { $ww^r$ |w belongs to {a, b}* } is CFL but not DCFL

❐❐❐

# 4 TURNING MACHINE

## 4.1 Classification of Languages




1. All languages which are not RELs
2. All not recursive languages
3. All not CSLs
4. All not CFLs
5. All not DCFLs
6. All not regulars
7. All recursive languages which are not DCFLs
8. All RELs which are not DCFLs

## 4.2 There are Two types of TM

(a) DTM (Deterministic TM)
(b) NTM (Non- Deterministic TM)

**DTM**

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \left\{ \underset{\text{Left}}{\downarrow} L, \underset{\text{Right}}{\downarrow} R \right\}$$

**NTM**

$$\delta : Q \times \Gamma \rightarrow 2^{Q \times \Gamma \times \{L, R\}}$$

## 4.3 Unrestricted Grammar (UG) and Context Sensitive Grammar (CSG)

Unrestricted grammar (UG) also called as

- RE grammar
- Phase structure grammar

Context sensitive grammar (CSG) is also called as

- Non contracting grammar
- Bound restricted grammar

## 4.4 Grammars

**Left Linear Grammar (LLG):** $V \rightarrow VT^* \mid T^*$

**Right Linear Grammar (RLG):** $V \rightarrow T^*V \mid T^*$

**Linear Grammar (LG):** $V \rightarrow T^*VT^* \mid T^*$

**Context Free Grammar (CFG):** $LHS \rightarrow RHS, \quad |LHS| \leq |RHS|$

**Unrestricted Grammar (UG):** $LHS \rightarrow RHS$

## 4.5 Equivalence of various TMs

TM ≅ Single tape TM

TM ≅ One-way infinite tape TM

TM ≅ Two-way infinite tape TM

TM ≅ Multi tape and multi head TM

TM ≅ Universal TM

TM ≅ Two stack PDA

TM ≅ Multi stack PDA

TM ≅ FA with two stacks

TM ≅ FA + R/W tape + Bidirectional head

## 4.6 Restrictions on TM

(1) If TM tape is read only tape, this TM accepts regular.

TM ≅ FA (TM with read only tape)

(2) If TM head is unidirectional then L(TM) = Regular.

(3) If TM tape is read only and unidirectional head then L(TM) = Regular.

(4) If TM always halts then L(TM) is recursive language.

(5) If TM always halts and uses linear bound tape then L(TM) is CSL.

## 4.7 LBA, HTM and TM

**HTM:** It is a TM that always halts for every input.
**TM:** It halts for every valid string and for invalid strings either halts at "non final state" or "never halts".
**LBA:** It is HTM but length of the tape we use linearly bounded.

- LBA accepts CSL languages.
- HTM accepts recursive languages.
- TM accepts Recursive Enumerable (RE) languages.

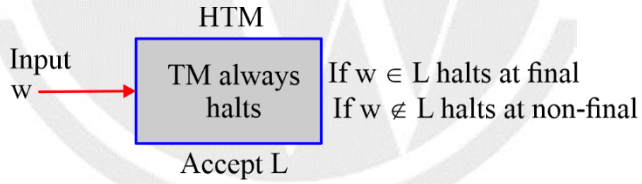
## 4.8 Recursively Enumerable Language

It is also called as:

- Enumerable language
- TM recognizable language
- TM Enumerable language
- Partially decidable language
- Semi-decidable language

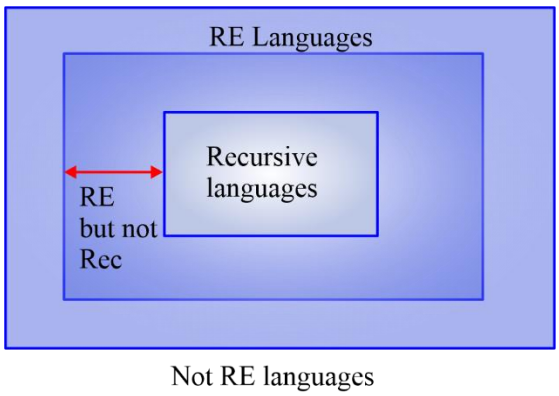
## 4.9 Recursive Language

- Recursive language is acceptable by HTM, and hence acceptable by TM.
- Recursive also called as decidable language.
- Recursive also called as Turing decidable language.
  If TM always halts, then TM is called as HTM.




## 4.10 Difference between Recursive and REL

- All Recursive languages are RE languages.



**Note :**

1. Union:
   - REL ∪ Finite ⇒ REL
   - REL ∪ Regular ⇒ REL
   - REL ∪ CFL ⇒ REL
   - REL ∪ Recursive ⇒ REL
   - $REL_1 \cup REL_2 \Rightarrow REL$
2. Intersection:
   - REL ∩ Finite ⇒ REL (Finite)
   - REL ∩ CFL ⇒ REL
   - REL ∩ Rec ⇒ REL
   - $REL_1 \cap REL_2 \Rightarrow REL$

## 4.11 Closure Properties of Recursive languages

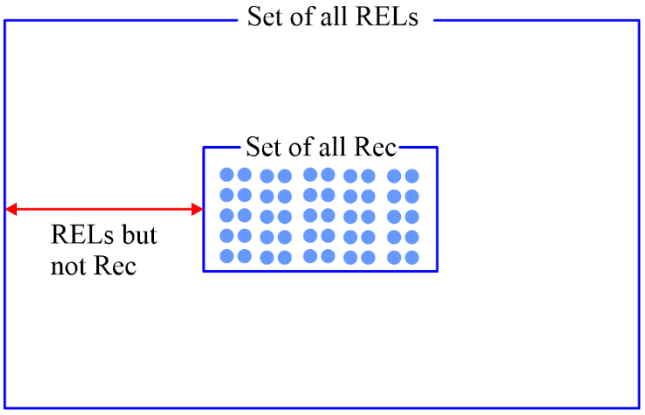
I. The following operations are not closed for recursive languages:

- Subset
- Substitution
- Homomorphism
- Finite substitution
- Infinite union
- Infinite intersection
- Infinite concatenation
- Infinite difference
- Infinite substitution

II. The following operations are closed for recursive languages:

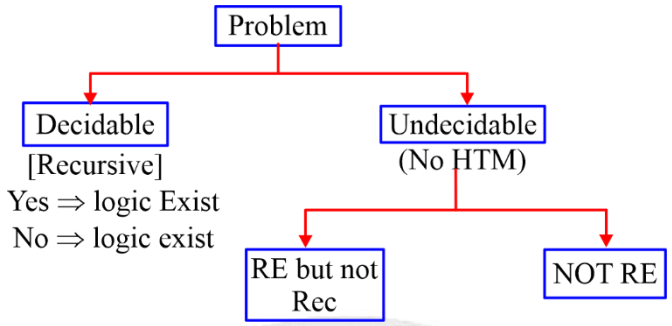
- Complement
- Difference
- Finite difference
- Infinite followed by ∪, ∩, -, •, ⊆, f
- Remember not closed operations

## 4.12 Complement of Recursive Vs Complement of REL



- Complement of Recursive set is Recursive.
- Complement of REL is either Recursive or non-REL.
- Complement of REL never be "REL which is not recursive".

## 4.13 Decidable and Undecidable




## 4.14 Decidable Vs Undecidable, RE Vs Not RE, Countable Vs Uncountable

[1] Decidable and Undecidable

↓ ↓

HTM exist (Rec language) | HTM not exist (not Rec language)

[2] RE and Not RE

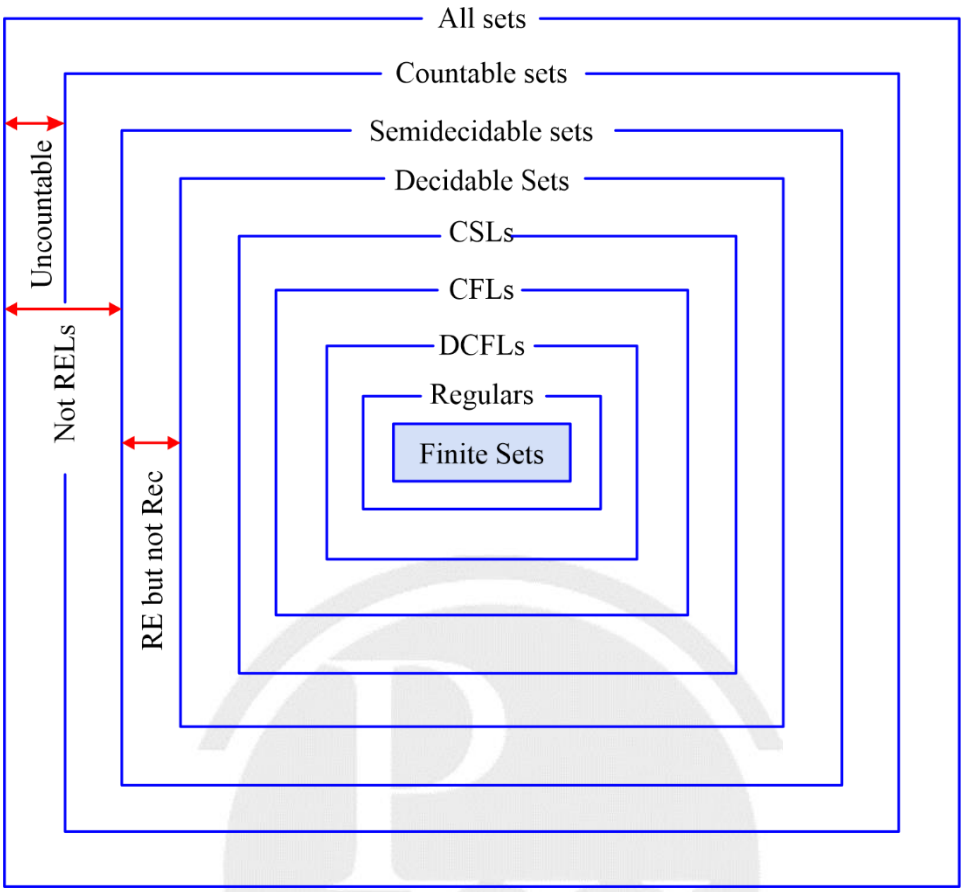
↓ ↓

TM exist | TM not exist

[3] Countable set and Uncountable set

↓ ↓

X is countable set iff $X \to f(X)$ is bijective where f(X) is known countable set. | (X is not countable set)




**Note :**

- If problem p is decidable then $\overline{p}$ is also decidable.
- If problem p is Undecidable then $\overline{p}$ is also UD.
- If problem p is RE but not recursive then $\overline{p}$ is not RE.
- If problem p is not RE then $\overline{p}$ is either "Not RE" or "RE but not Recursive".

## 4.15 Decision Properties Table

- D: Decidable
- UD: Undecidable

| | FA | DPDA | PDA | LBA/HTM | TM |
|---|---|---|---|---|---|
| H (Halting) | D | D | D | D | UD |
| M (Membership) | D | D | D | D | UD |
| $E_m$ (Emptiness) | D | D | D | UD | UD |
| F (Finiteness) | D | D | D | UD | UD |

| | | | | | |
|---|---|---|---|---|---|
| T (Totality) | D | D | UD | UD | UD |
| $E_q$ (Equivalence) | D | D | UD | UD | UD |
| D (Disjoint) | D | UD | UD | UD | UD |
| S (Set Containment) | D | UD | UD | UD | UD |

## 4.16 Decidable problem for DFA / NFA/ FA/ Regular

(1) Halting problem for FA / Reg/ DFA / NFA is decidable.
(2) Non-halting problem for FA / Reg/ DFA / NFA is decidable.
(3) Membership problem for FA / Reg/ DFA / NFA is decidable.
(4) Non-membership problem for FA / Reg/ DFA / NFA is decidable.
(5) Emptiness for FA / Reg/ DFA / NFA is decidable.
(6) Non-emptiness problem for FA / Reg/ DFA / NFA is decidable.
(7) Fitness problem for FA / Reg/ DFA / NFA is decidable.
(8) Non-fitness problem for FA / Reg/ DFA / NFA is decidable.
(9) Totality problem for FA / Reg/ DFA / NFA is decidable.
(10) Non-totality problem for FA / Reg/ DFA / NFA is decidable.
(11) Equivalence problem for FA / Reg/ DFA / NFA is decidable.
(12) Non-equivalence problem for FA / Reg/ DFA / NFA is decidable.
(13) Disjointness problem is decidable for FA / Reg/ DFA / NFA.
(14) N0n-disjointness problem is decidable for FA / Reg/ DFA / NFA.
(15) Set containment problem for FA / Reg/ DFA / NFA is decidable
(16) Non-set containment problem for FA / Reg/ DFA / NFA is decidable.

## 4.17 Decidable problems for CFLs/DCFLs

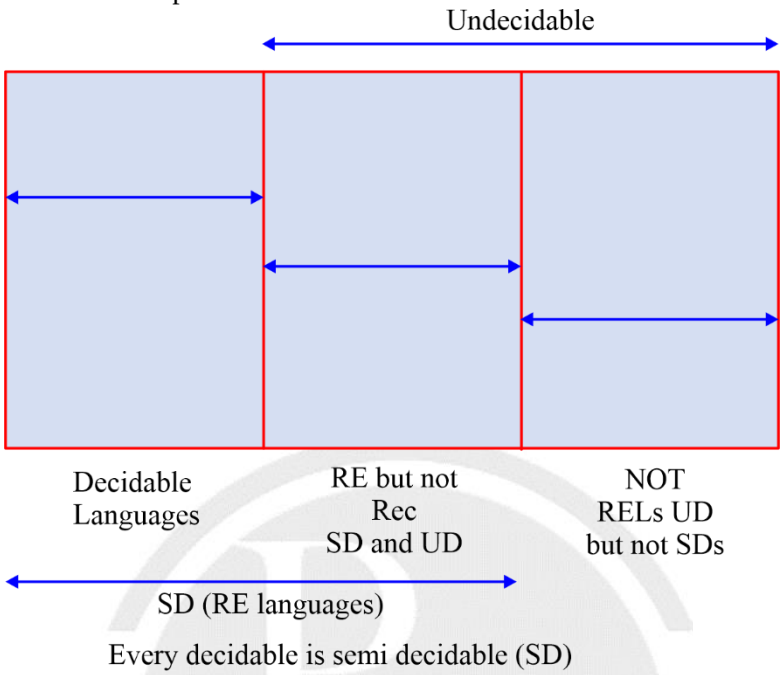
| | Problems | DCFLs | CFLs |
|---|---|---|---|
| | Halting | D | D |
| | Non-Halting | D | D |
| CYK algo | Membership | D | D |
| CYK algo | Non-membership | D | D |
| Simplification algo | Emptiness | D | D |
| Simplification algo | Non-emptiness | D | D |
| Dependancy graph | Finiteness | D | D |
| Dependancy graph | Non-finiteness | D | D |
| | Totality | D | UD |
| | Non-totality | D | UD |
| | Equivalence | D | UD |
| | Disjointness | UD | UD |
| | Non-disjointness | UD | UD |
| | Set containment | UD | UD |
| | Non-set containment | UD | UD |

## 4.18 Decidability problems for Recursive languages

| | **Problems** | **Recursive** |
|---|---|---|
| 1. | Halting | D |
| 2. | Non-Halting | D |
| 3. | Membership | D |
| 4. | Non-membership | D |
| 5. | Emptiness | UD [Not REL] |
| 6. | Non-emptiness | UD [RE but not Rec] [SD but UD] |
| 7. | Finiteness | UD [Not RE] |
| 8. | Non-finiteness | UD [Not RE] |
| 9. | Totality | UD [Not RE] |
| 10. | Non-totality | UD [RE but not Rec] [SD but UD] |
| 11. | Equivalence | UD [Not RE] |
| 12. | Non-equivalence | UD [RE but not Rec] |
| 13. | Disjointness | UD [Not RE] |
| 14. | Non-disjointness | UD [RE but not Rec] |
| 15. | Set containment | UD [Not RE] |
| 16. | Non-set containment | UD [RE but not Rec] |

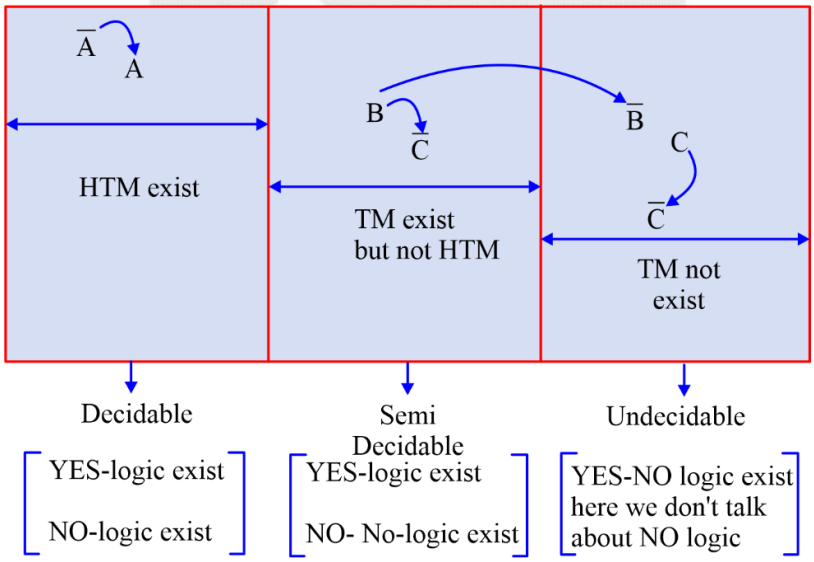
## 4.19 Classification of Languages based on Decidability

All RE languages can be classified into 3 important classes.




Every decidable is semi decidable (SD)

### 4.19.1 Decidability Vs Turing Machine




## 4.20 Decidable languages

(1) Finite set $\Rightarrow$ Decidable
(2) $\sum = \{a, b\} \Rightarrow$ Decidable
(3) $\sum \cong$ Set of finite number of symbols $\Rightarrow$ Decidable
(4) $\sum^*$ over alphabet $\sum = \{a, b\} \Rightarrow$ Decidable
(5) $\{M \mid M$ is DFA, M accepts $ab\} \Rightarrow$ Decidable

(6) {TM | Number of states in TM= 2} ⇒ Decidable

(7) {TM | TM reaches state q within 100 steps} ⇒ Decidable

(8) { TM | TM accepts REL} ⇒ Decidable

❐❐❐❐